



《模式识别》

第十二章 Transformer

马锦华

<https://cse.sysu.edu.cn/teacher/MaJinhua>

SUN YAT-SEN University



声明：该PPT只供非商业使用，也不可视为任何出版物。由于历史原因，许多图片尚没有标注出处，如果你知道图片的出处，欢迎告诉我们 majh8@mail.sysu.edu.cn.

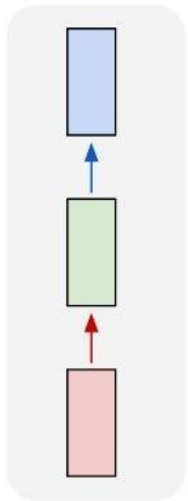


课程目录（暂定）

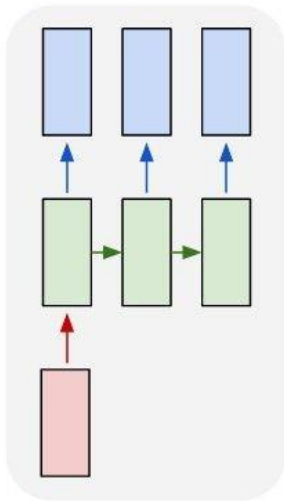
❑	第一章	课程简介与预备知识	6学时
❑	第二章	特征提取与表示	6学时
❑	第三章	主成分分析	3学时
❑	第四章	归一化、判别分析、人脸识别	3学时
❑	第五章	EM算法与聚类	3学时
❑	第六章	贝叶斯决策理论	3学时
❑	第七章	线性分类器与感知机	3学时
❑	第八章	支持向量机	3学时
❑	第九章	神经网络、正则项和优化方法	3学时
❑	第十章	卷积神经网络及经典框架	6学时
❑	第十一章	循环神经网络	2学时
❑	第十二章	Transformer	2学时
❑	第十三章	自监督与半监督学习	2学时
❑	第十四章	开放世界模式识别	2学时

Last Time: Recurrent Neural Networks

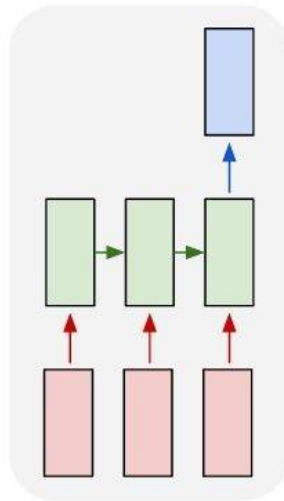
one to one



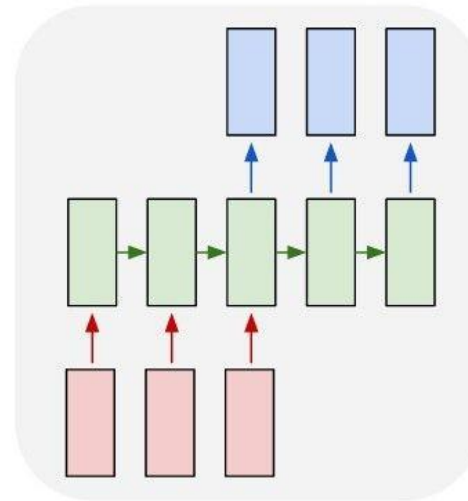
one to many



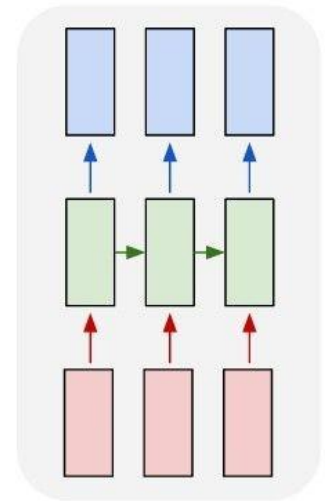
many to one



many to many



many to many



Last Time: Variable length computation graph with shared weights

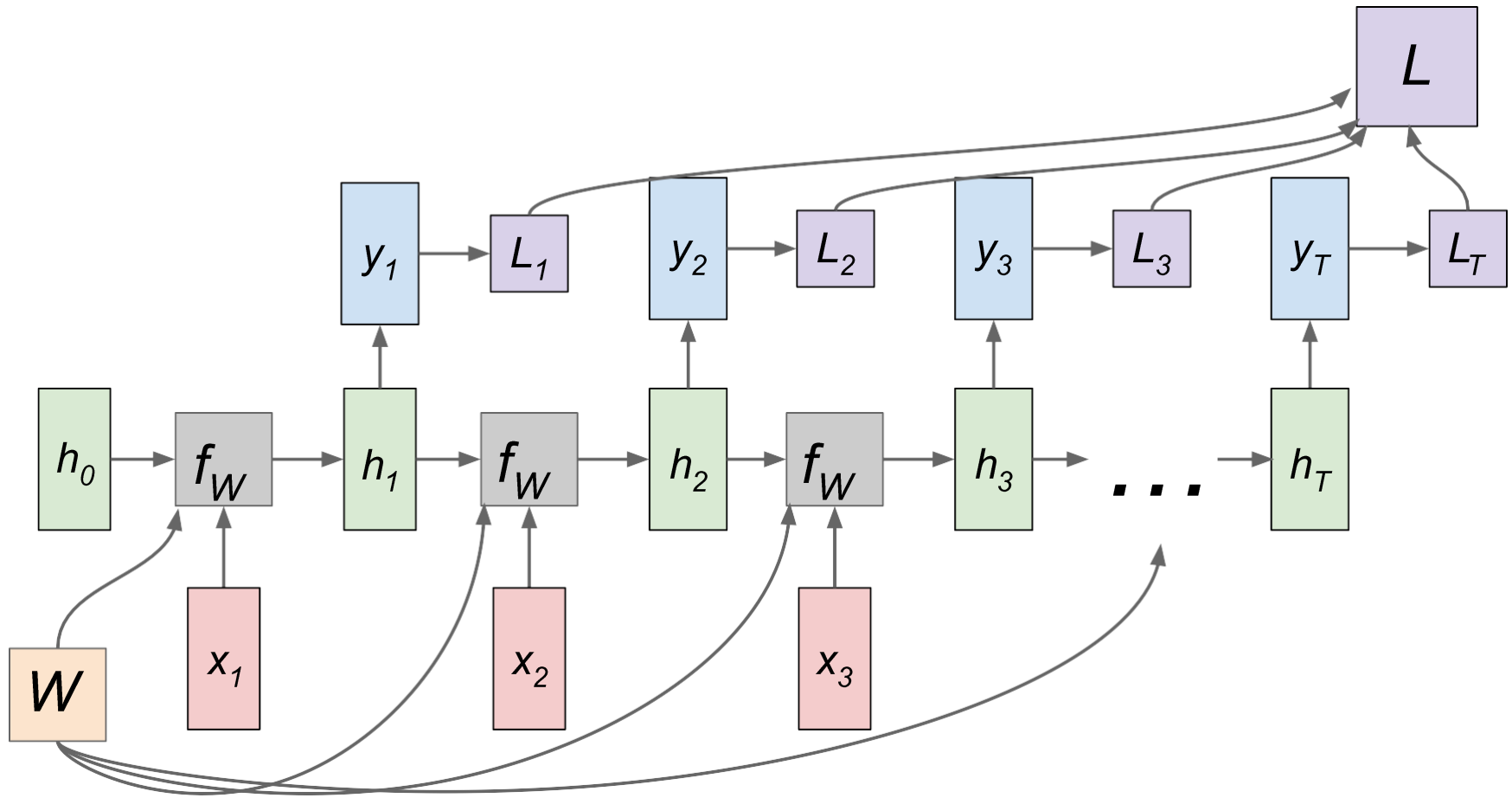
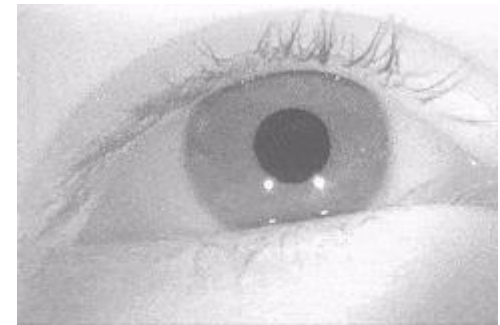
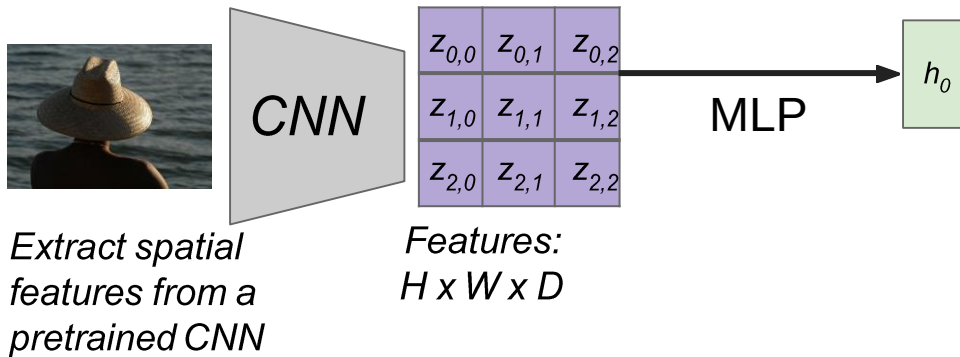


Image Captioning with RNNs and Attention

- Attention idea: New context vector at every time step.
- Each context vector will attend to different image regions

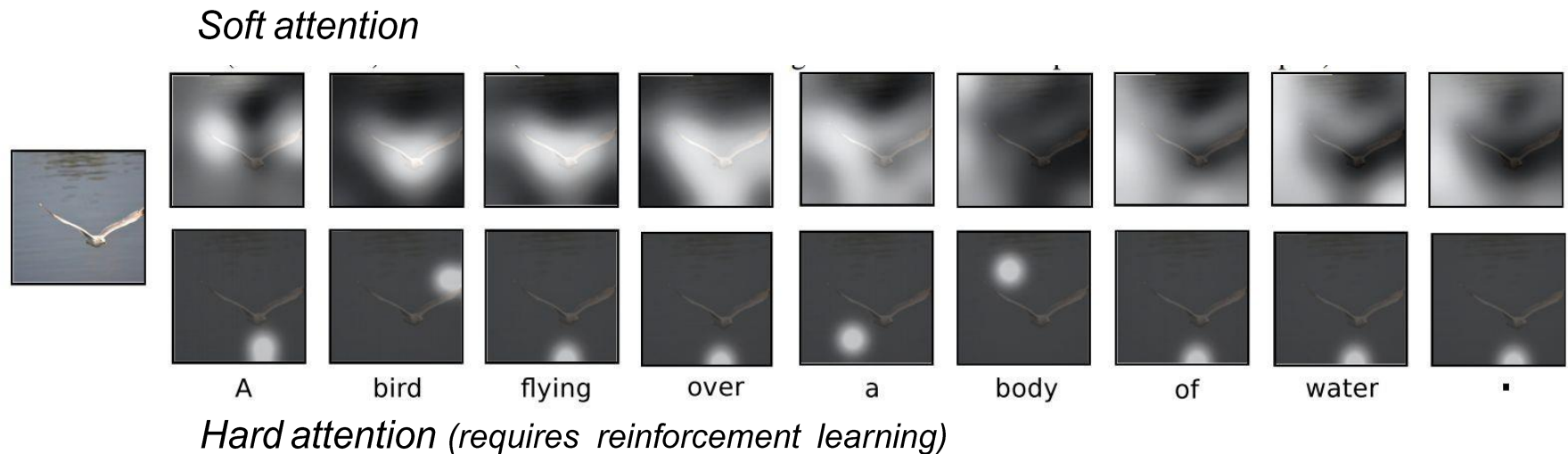
[gif source](#)



Attention Saccades in humans

Image Captioning with Attention

- Comparison



Xu et al, "Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Figure copyright Kelvin Xu, Jimmy Lei Ba, Jamie Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhutdinov, Richard S. Zemel, and Yoshua Bengio, 2015. Reproduced with permission.

Image Captioning with Attention

- Visualization



A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.



A giraffe standing in a forest with trees in the background.

Xu et al, "Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Figure copyright Kelvin Xu, Jimmy Lei Ba, Jamie Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhutdinov, Richard S. Zemel, and Yoshua Bengio, 2015. Reproduced with permission.

Attention we just saw in image captioning

Features

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

h

Inputs:

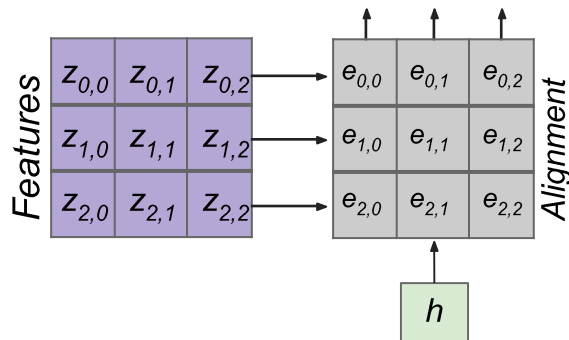
Features: $\mathbf{z}$ (shape: $H \times W \times D$)

Query: $\mathbf{h}$ (shape: D)

Attention we just saw in image captioning

Operations:

$$\text{Alignment: } e_{i,j} = f_{\text{att}}(h, z_{i,j})$$

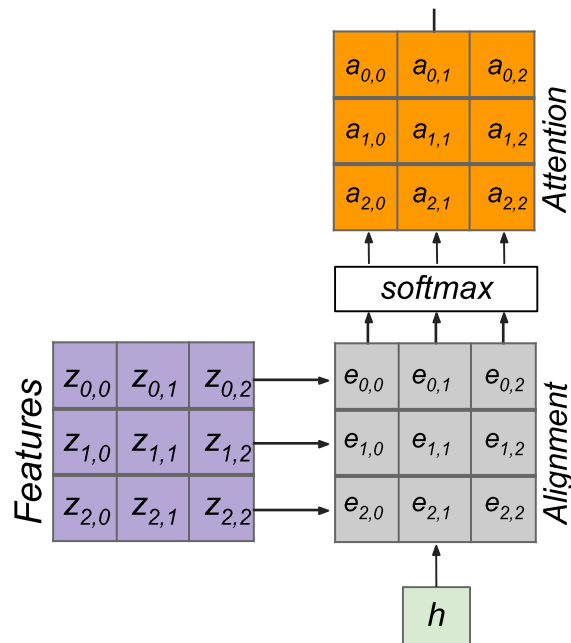


Inputs:

Features: $\mathbf{z}$ (shape: $H \times W \times D$)

Query: $\mathbf{h}$ (shape: D)

Attention we just saw in image captioning



Operations:

Alignment: $e_{i,j} = f_{\text{att}}(h, z_{i,j})$

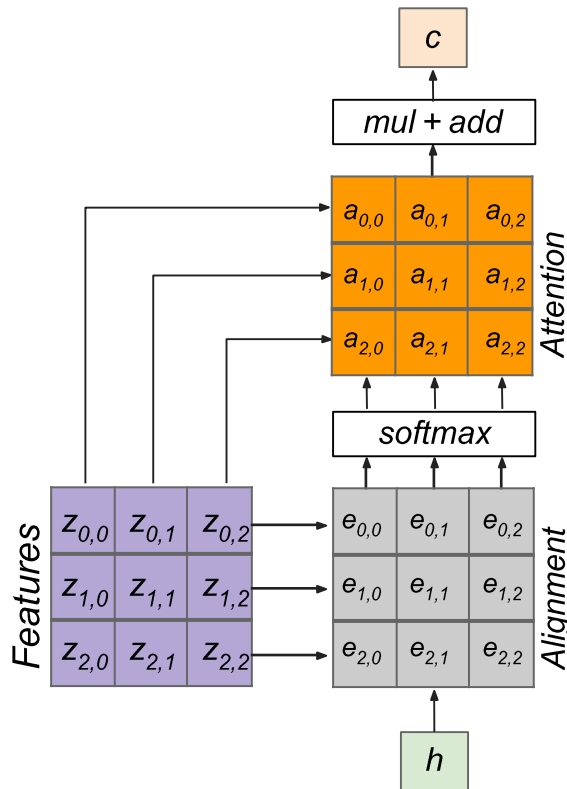
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Inputs:

Features: $\mathbf{z}$ (shape: $H \times W \times D$)

Query: $\mathbf{h}$ (shape: D)

Attention we just saw in image captioning



Outputs:

context vector: $\mathbf{c}$ (shape: D)

Operations:

Alignment: $e_{i,j} = f_{att}(h, z_{i,j})$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

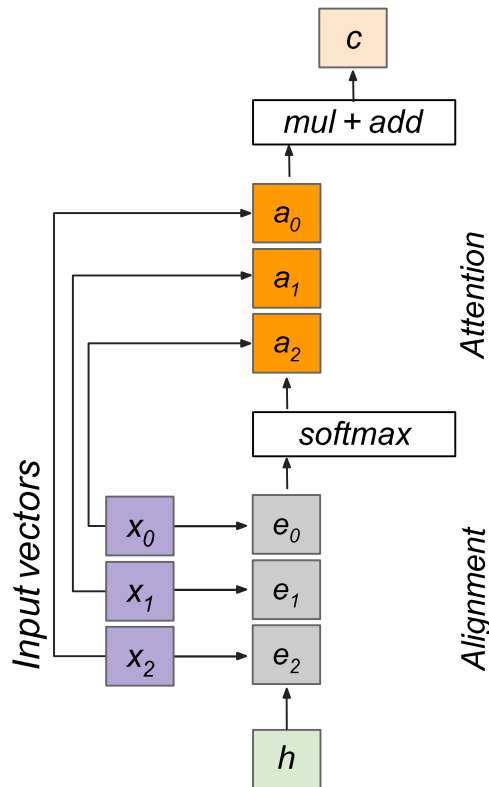
Output: $\mathbf{c} = \sum_{i,j} a_{i,j} z_{i,j}$

Inputs:

Features: $\mathbf{z}$ (shape: $H \times W \times D$)

Query: $\mathbf{h}$ (shape: D)

General attention layer



Outputs:
context vector: $\mathbf{c}$ (shape: D)

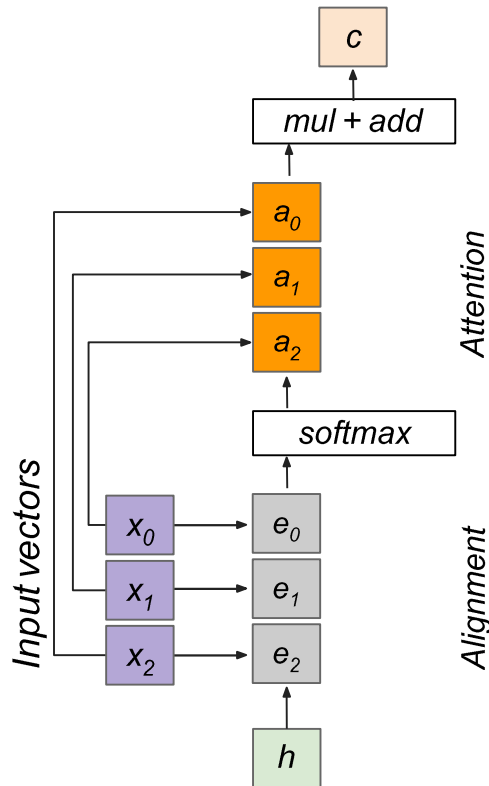
Operations:
Alignment: $e_i = f_{\text{att}}(h, x_i)$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $\mathbf{c} = \sum_i a_i x_i$

Inputs:
Input vectors: $\mathbf{x}$ (shape: $N \times D$)
Query: $\mathbf{h}$ (shape: D)

*Attention operation is **permutation invariant**.*

- *Doesn't care about ordering of the features*
- *Stretch $\mathbf{H} \times \mathbf{W} = \mathbf{N}$ into N vectors*

General attention layer



Outputs:
context vector: $\mathbf{c}$ (shape: D)

Operations:

Alignment: $e_i = h \cdot x_i$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $\mathbf{c} = \sum_i a_i x_i$

Change $f_{\text{att}}(.)$ to a simple dot product

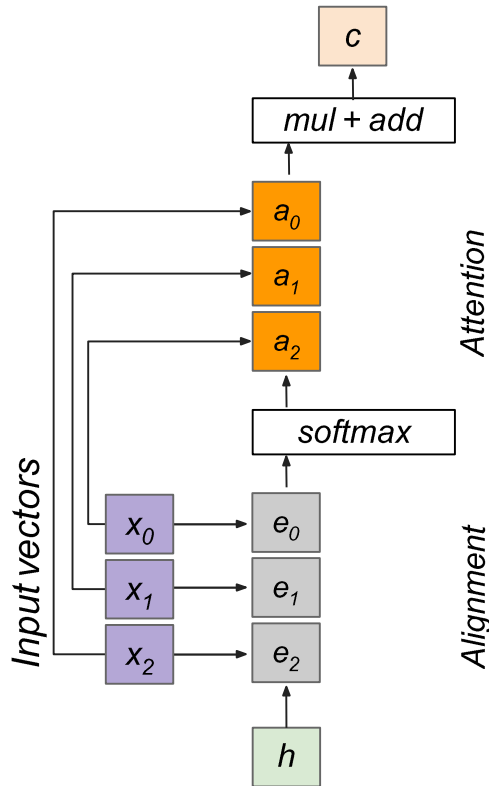
- only works well with key & value transformation trick (will mention in a few slides)

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Query: $\mathbf{h}$ (shape: D)

General attention layer



Outputs:
context vector: $\mathbf{c}$ (shape: D)

Operations:

Alignment: $e_i = h \cdot x_i / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $\mathbf{c} = \sum_i a_i x_i$

Inputs:

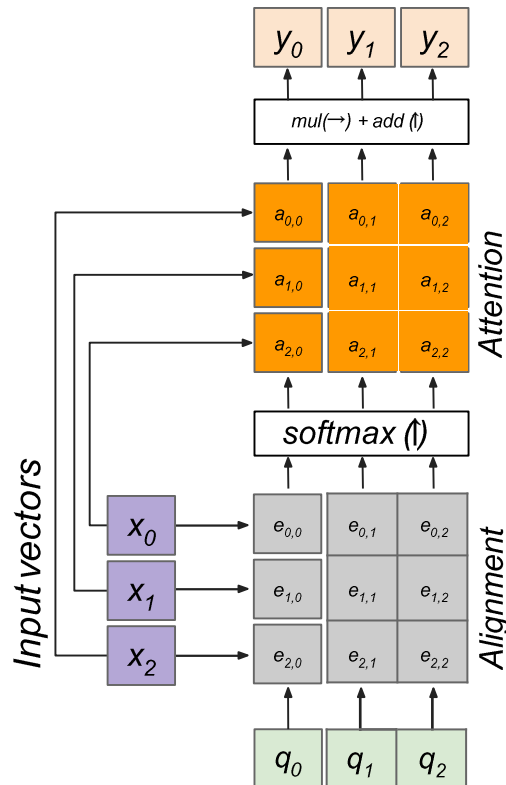
Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Query: $\mathbf{h}$ (shape: D)

Change $f_{\text{att}}(\cdot)$ to a **scaled** simple dot product

- Larger dimensions means more terms in the dot product sum.
- So, the variance of the logits is higher.
Large magnitude vectors will produce much higher logits.
- So, the post-softmax distribution has lower-entropy, assuming logits are IID.
- Ultimately, these large magnitude vectors will cause softmax to peak and assign very little weight to all others
- Divide by $\sqrt{D}$ to reduce effect of large magnitude vectors

General attention layer



Outputs:

context vectors: $\mathbf{y}$ (shape: D)

Operations:

Alignment: $e_{i,j} = q_j \cdot x_i / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $y_j = \sum_i a_{i,j} x_i$

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

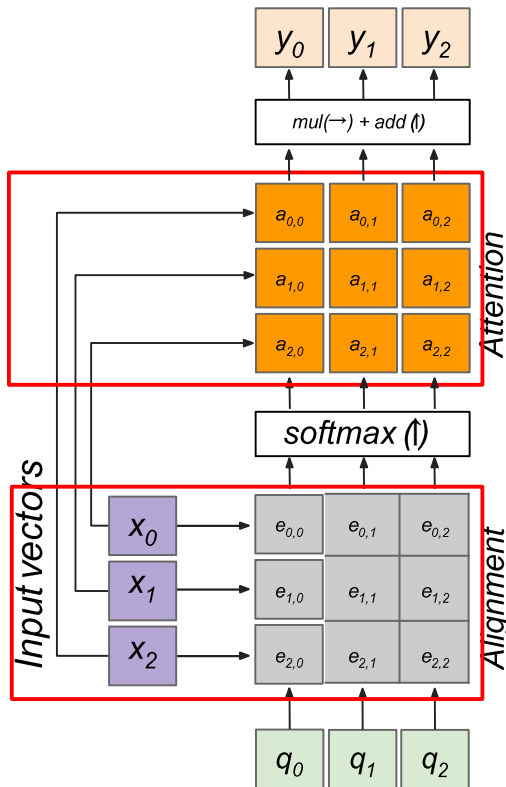
Queries: $\mathbf{q}$ (shape: $M \times D$)

Multiple context vectors

- each query creates a new output context vector

Multiple query vectors

General attention layer



Outputs:
context vectors: $\mathbf{y}$ (shape: D)

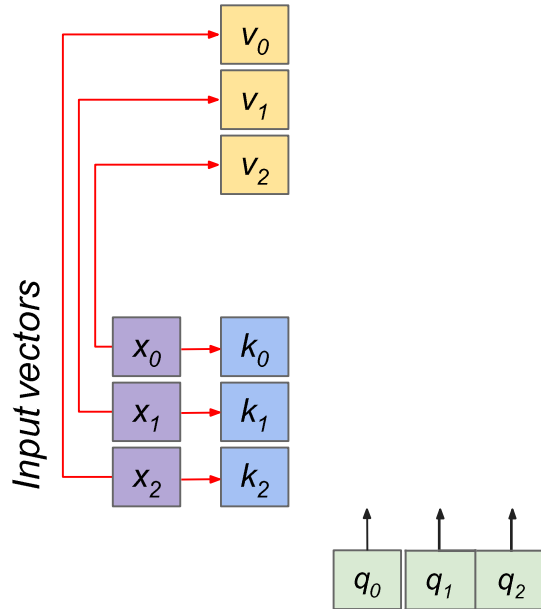
Operations:
Alignment: $e_{i,j} = q_j \cdot x_i / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} x_i$

Inputs:
Input vectors: $\mathbf{x}$ (shape: $N \times D$)
Queries: $\mathbf{q}$ (shape: $M \times D$)

Notice that the input vectors are used for both the alignment as well as the attention calculations.

- We can add more expressivity to the layer by adding a different FC layer before each of the two steps.

General attention layer



Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$

Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

Notice that the input vectors are used for both the alignment as well as the attention calculations.

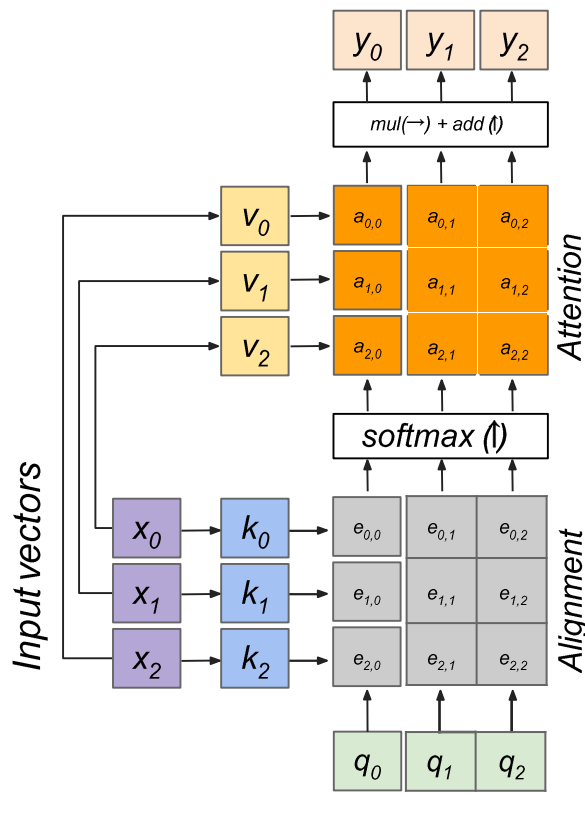
- We can add more expressivity to the layer by adding a different FC layer before each of the two steps.

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Queries: $\mathbf{q}$ (shape: $M \times D_k$)

General attention layer



Outputs:

context vectors: $\mathbf{y}$ (shape: $\boxed{D_v}$)

The input and output dimensions can now change depending on the key and value FC layers

Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$

Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $y_j = \sum_i a_{i,j} v_i$

Notice that the input vectors are used for both the alignment as well as the attention calculations.

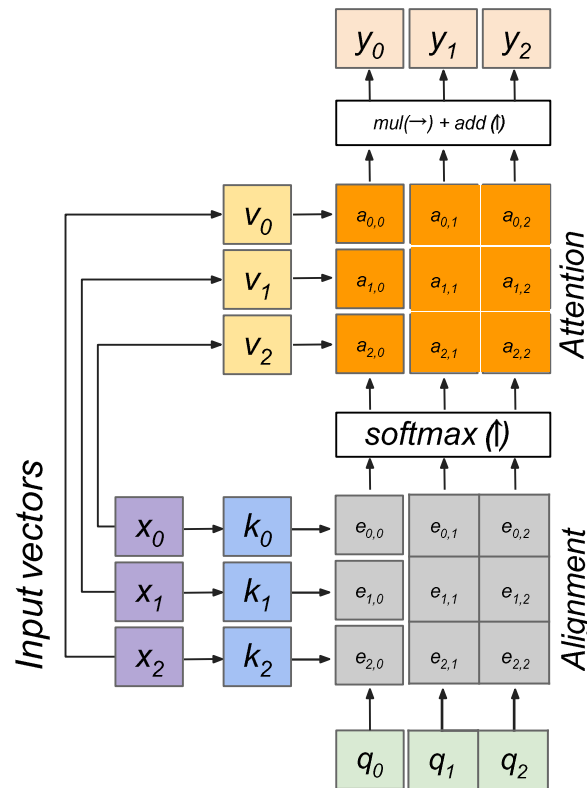
- *We can add more expressivity to the layer by adding a different FC layer before each of the two steps.*

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Queries: $\mathbf{q}$ (shape: $M \times \boxed{D_k}$)

Self attention layer



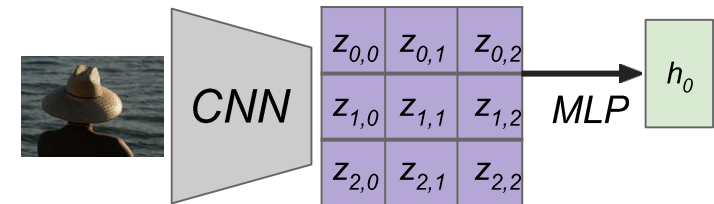
Outputs:
context vectors: $\mathbf{y}$ (shape: D_v)

Operations:
Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$
Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$
Alignment: $e_{i,j} = q_i \cdot k_j / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} v_i$

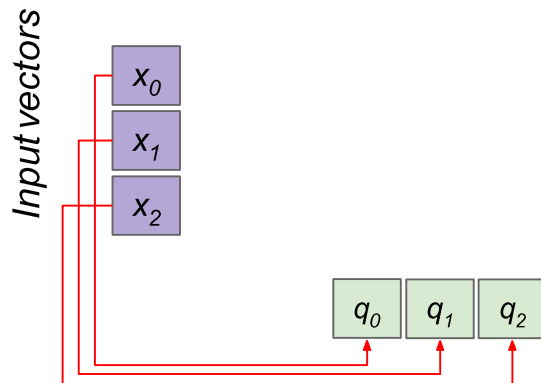
Inputs:
Input vectors: $\mathbf{x}$ (shape: $N \times D$)
Queries: $\mathbf{q}$ (shape: $M \times D_k$)

Recall that the query vector was a function of the input vectors.

Encoder: $h_0 = f_w(\mathbf{z})$
where $\mathbf{z}$ is spatial CNN features
 $f_w(\cdot)$ is an MLP



Self attention layer



Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$

Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

Query vectors: $\mathbf{q} = \mathbf{x}\mathbf{W}_q$

Alignment: $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $y_i = \sum_j a_{ij} v_j$

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

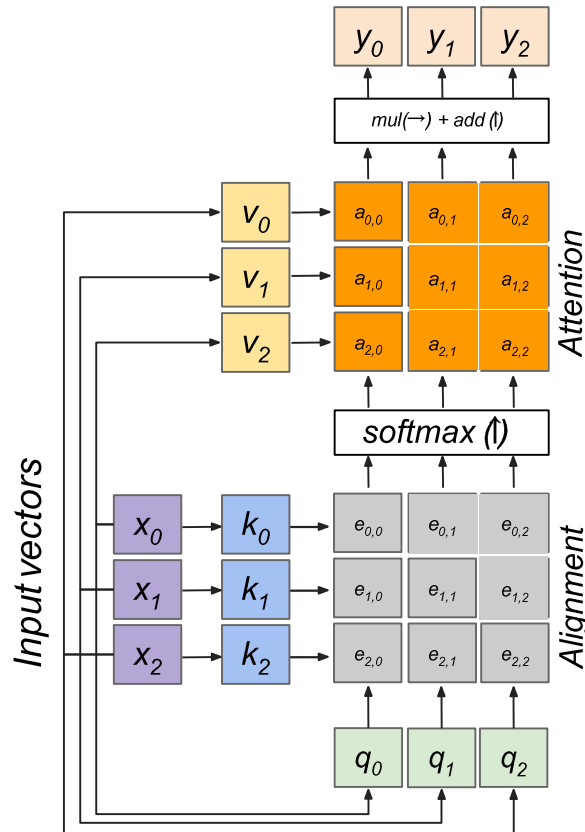
Queries: $\mathbf{q}$ (shape: $M \times D_k$)

We can calculate the query vectors from the input vectors, therefore, defining a "self-attention" layer.

Instead, query vectors are calculated using a FC layer.

No input query vectors anymore

Self attention layer



Outputs:

context vectors: $\mathbf{y}$ (shape: D_v)

Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$

Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

Query vectors: $\mathbf{q} = \mathbf{x}\mathbf{W}_q$

Alignment: $e_{i,j} = q_i \cdot k_j / \sqrt{D}$

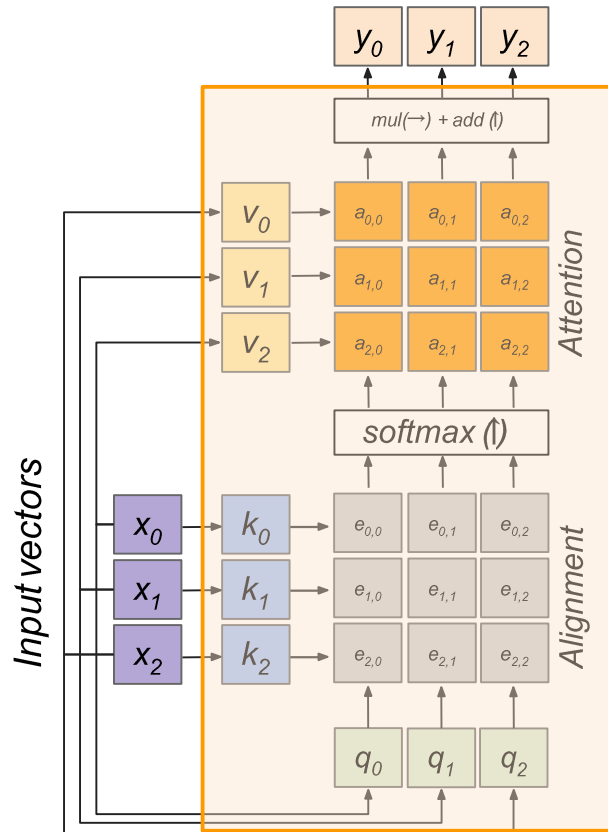
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

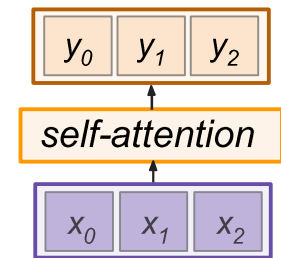
Self attention layer - attends over sets of inputs



Outputs:
context vectors: y (shape: D_v)

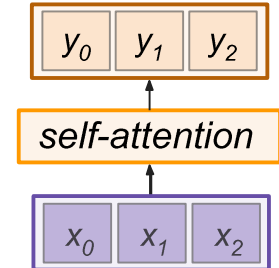
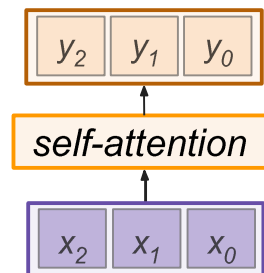
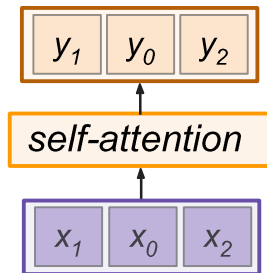
Operations:
Key vectors: $k = xW_k$
Value vectors: $v = xW_v$
Query vectors: $q = xW_q$
Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
Attention: $a = \text{softmax}(e)$
Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:
Input vectors: x (shape: $N \times D$)

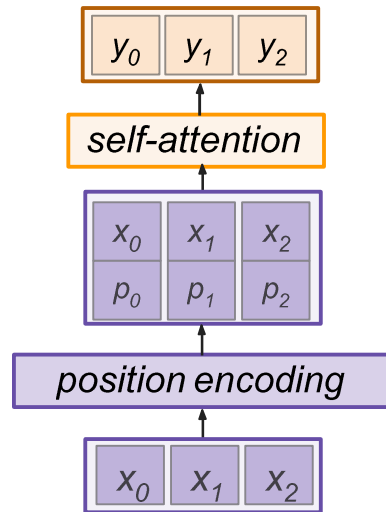


Self attention layer - attends over sets of inputs

- Permutation equivariant
- Self-attention layer doesn't care about the orders of the inputs!
- **Problem:** How can we encode ordered sequences like language or spatially ordered image features?



Positional encoding



Concatenate/add special positional encoding p_j to each input vector x_j

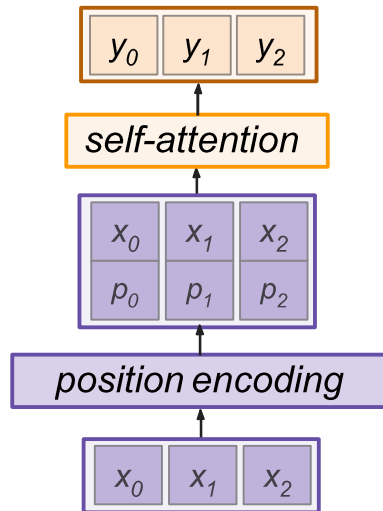
We use a function $\text{pos}: N \rightarrow R^d$ to process the position j of the vector into a d -dimensional vector

So, $p_j = \text{pos}(j)$

Desiderata of $\text{pos}(\cdot)$:

1. It should output a **unique** encoding for each time-step (word's position in a sentence)
2. **Distance** between any two time-steps should be consistent across sentences with different lengths.
3. Our model should generalize to **longer** sentences without any efforts. Its values should be bounded.
4. It must be **deterministic**.

Positional encoding



Concatenate special positional encoding p_i to each input vector x_i

We use a function $pos: N \rightarrow R^d$ to process the position j of the vector into a d -dimensional vector
So, $p_j = pos(j)$

Options for $pos(\cdot)$

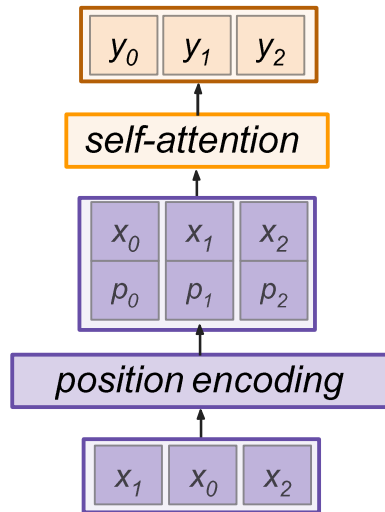
1. Learn a lookup table:

- Learn parameters to use for $pos(t)$ for $t \in [0, T)$
- Lookup table contains $T \times d$ parameters.

Desiderata of $pos(\cdot)$:

1. It should output a **unique** encoding for each time-step (word's position in a sentence)
2. **Distance** between any two time-steps should be consistent across sentences with different lengths.
3. Our model should generalize to **longer** sentences without any efforts. Its values should be bounded.
4. It must be **deterministic**.

Positional encoding



Concatenate special positional encoding p_j to each input vector x_j

We use a function $\text{pos}: N \rightarrow R^d$ to process the position j of the vector into a d -dimensional vector

So, $p_j = \text{pos}(j)$

Options for $\text{pos}(\cdot)$

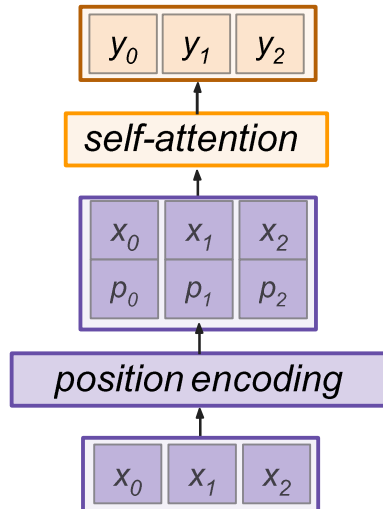
1. Learn a lookup table:
 - Learn parameters to use for $\text{pos}(t)$ for $t \in [0, T)$
 - Lookup table contains $T \times d$ parameters.
2. Design a fixed function with the desiderata

$$\mathbf{p}(t) = \begin{bmatrix} \sin(\omega_1 \cdot t) \\ \cos(\omega_1 \cdot t) \\ \sin(\omega_2 \cdot t) \\ \cos(\omega_2 \cdot t) \\ \vdots \\ \sin(\omega_{d/2} \cdot t) \\ \cos(\omega_{d/2} \cdot t) \end{bmatrix}_d$$

$$\vec{p}_t^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k \cdot t), & \text{if } i = 2k \\ \cos(\omega_k \cdot t), & \text{if } i = 2k + 1 \end{cases}$$

$$\text{where } \omega_k = \frac{1}{10000^{2k/d}}$$

Positional encoding



Concatenate special positional encoding p_i to each input vector x_i

We use a function $pos: N \rightarrow R^d$ to process the position j of the vector into a d -dimensional vector

Options for $pos(\cdot)$

1. Learn a lookup table:
 - Learn parameters to use for $pos(t)$ for $t \in [0, T]$
 - Lookup table contains $T \times d$ parameters.
2. Design a fixed function with the desiderata

$$p(t) = \begin{bmatrix} \sin(\omega_1 \cdot t) \\ \cos(\omega_1 \cdot t) \\ \vdots \\ \sin(\omega_{d/2} \cdot t) \\ \cos(\omega_{d/2} \cdot t) \end{bmatrix}_d$$

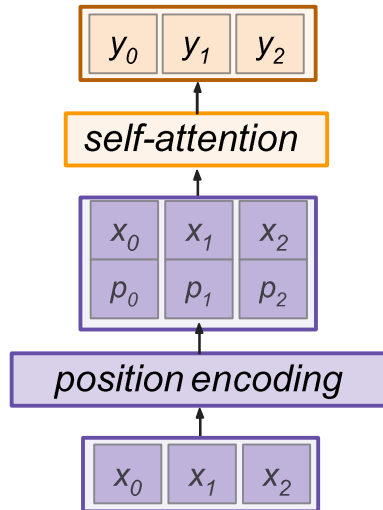
Intuition:

0:	0	0	0	0	8:	1	0	0	0
1:	0	0	0	1	9:	1	0	0	1
2:	0	0	1	0	10:	1	0	1	0
3:	0	0	1	1	11:	1	0	1	1
4:	0	1	0	0	12:	1	1	0	0
5:	0	1	0	1	13:	1	1	0	1
6:	0	1	1	0	14:	1	1	1	0
7:	0	1	1	1	15:	1	1	1	1

where $\omega_k = \frac{1}{10000^{2k/d}}$

[image source](#)

Positional encoding



Concatenate special positional encoding p_i to each input vector x_i

We use a function $pos: N \rightarrow R^d$ to process the position j of the vector into a d -dimensional vector

Visualization

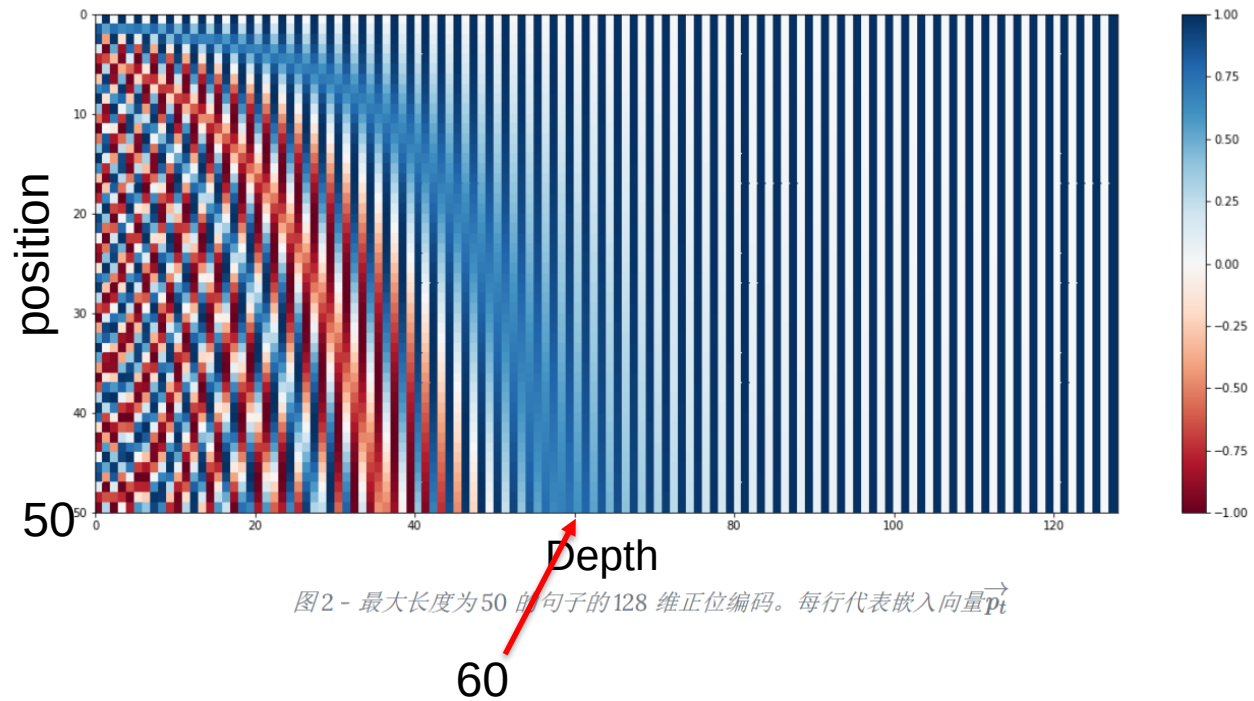
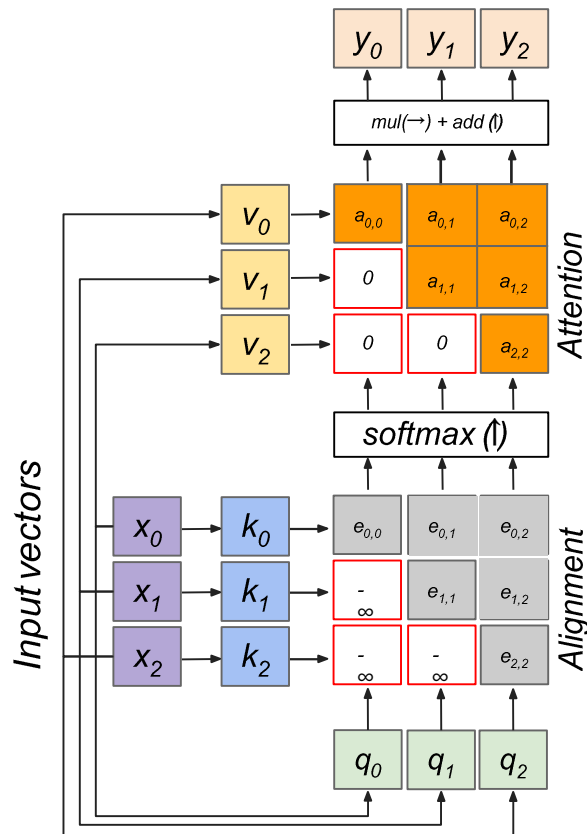


图2 - 最大长度为50 的句子128 维正位编码。每行代表嵌入向量 $\vec{p}_t$

Masked self-attention layer



Outputs:

context vectors: $\mathbf{y}$ (shape: D_v)

Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$

Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

Query vectors: $\mathbf{q} = \mathbf{x}\mathbf{W}_q$

Alignment: $e_{i,j} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $y_j = \sum_i a_{i,j} v_i$

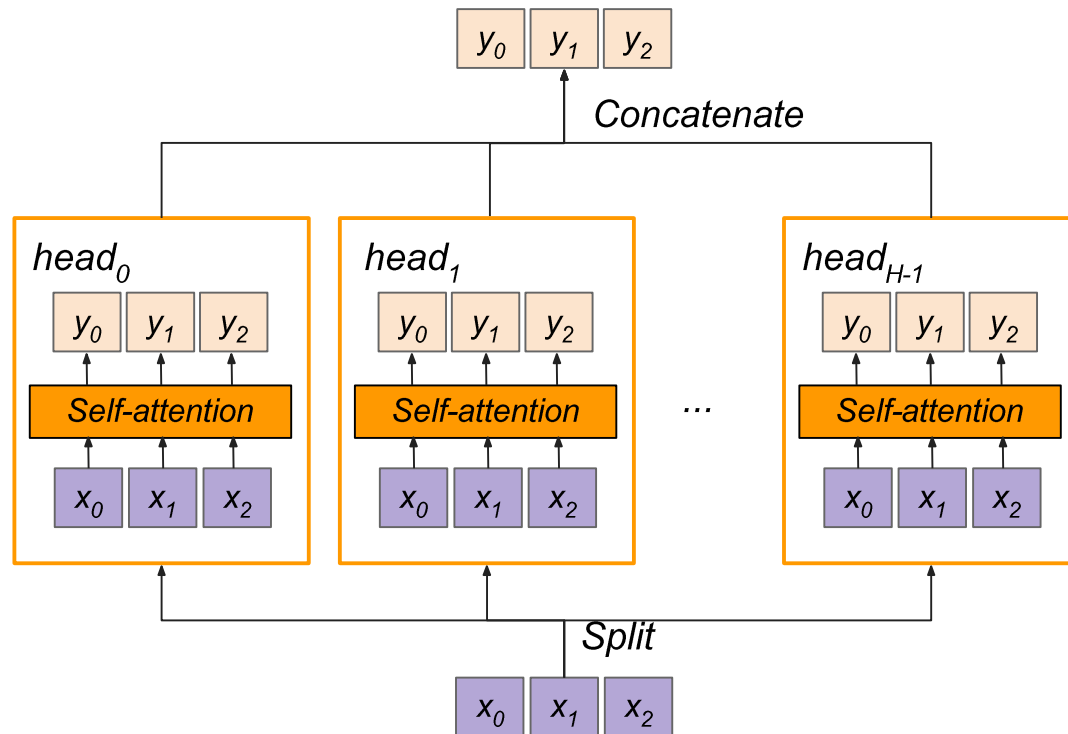
Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

- Prevent vectors from looking at future vectors.
- Manually set alignment scores to -infinity

Multi-head self attention layer

- - Multiple self-attention heads in parallel



Multi-head self attention layer

- - visualization

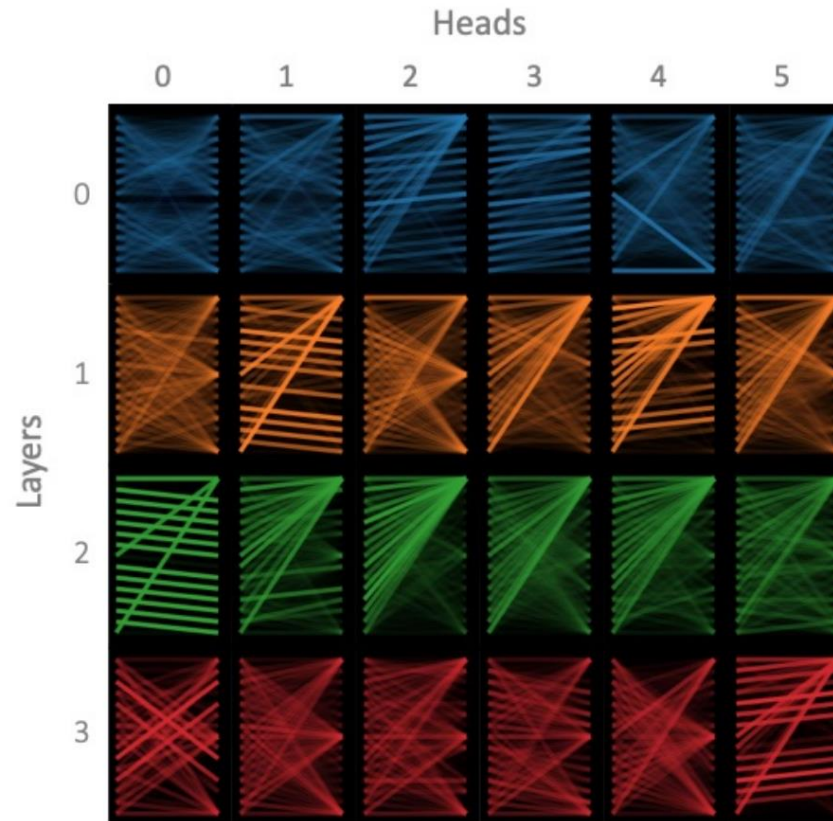
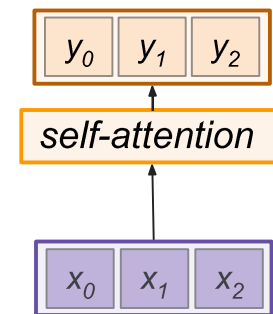
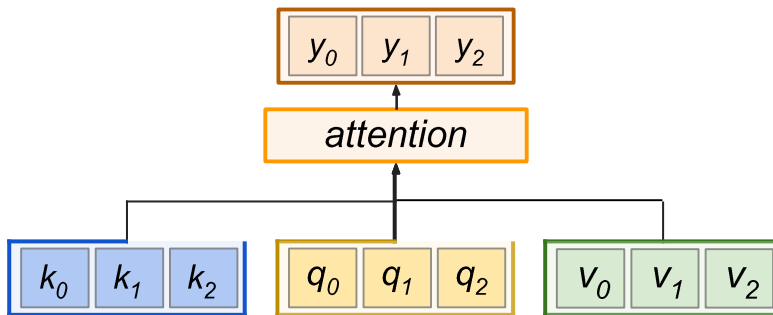


Figure 5: Model view of BERT, for same inputs as in Figure 2. Excludes layers 4–11 and heads 6–11.

Vig, Jesse. "A Multiscale Visualization of Attention in the Transformer Model." *ArXiv abs/1906.05714* (2019): n. pag.

General attention versus self-attention



Example: CNN with Self-Attention

Input Image

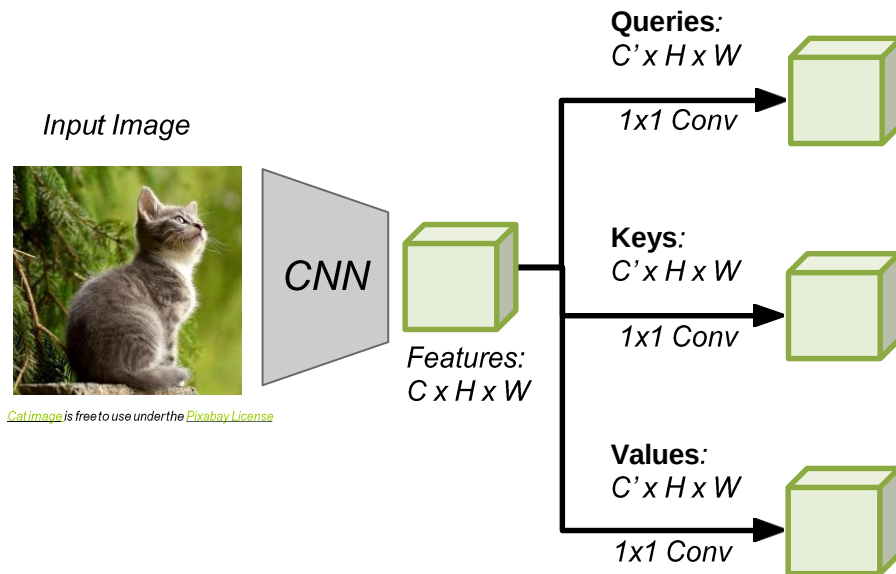


[Cat image](#) is free to use under the [Pixabay License](#)

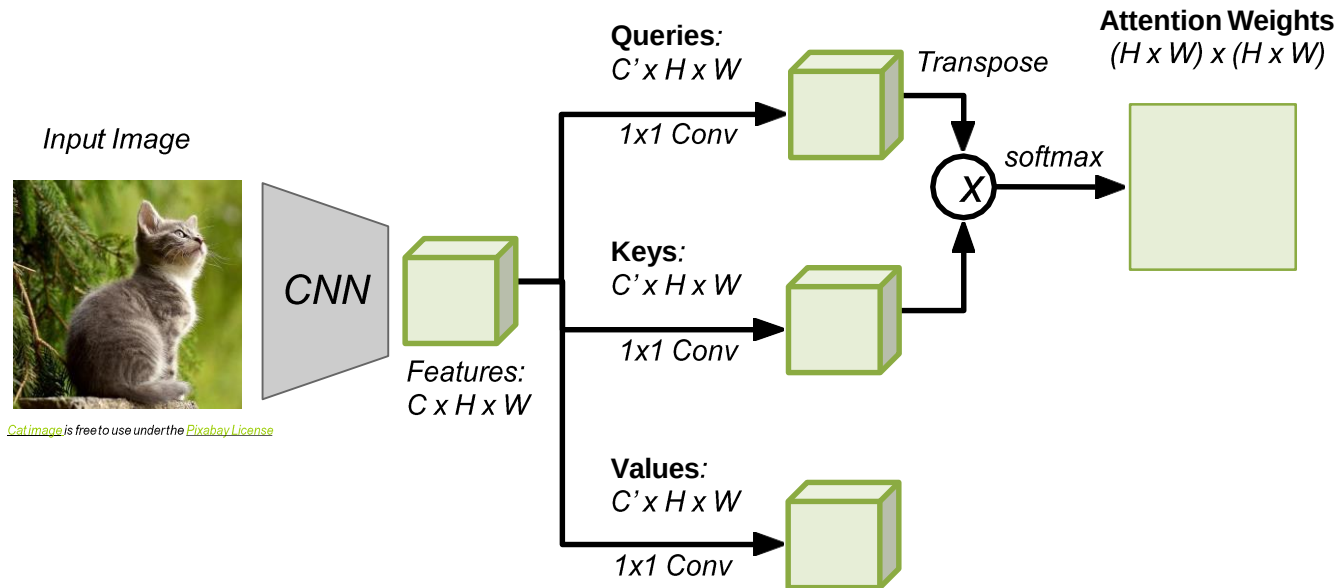


Features:
 $C \times H \times W$

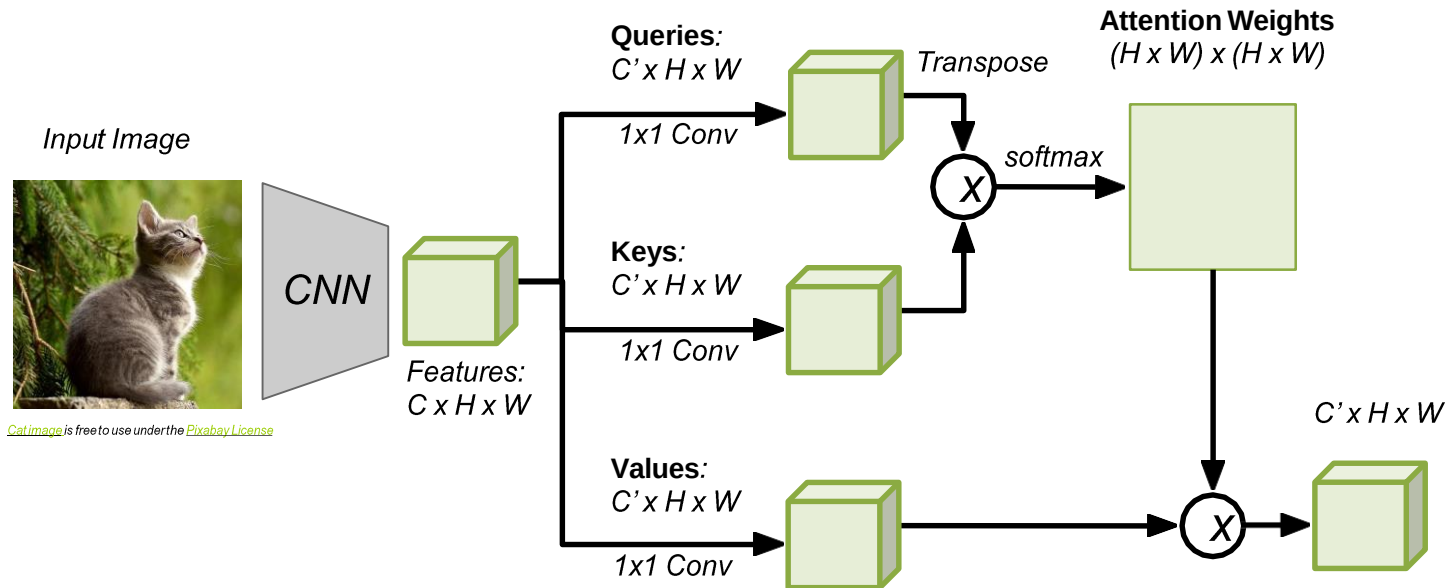
Example: CNN with Self-Attention



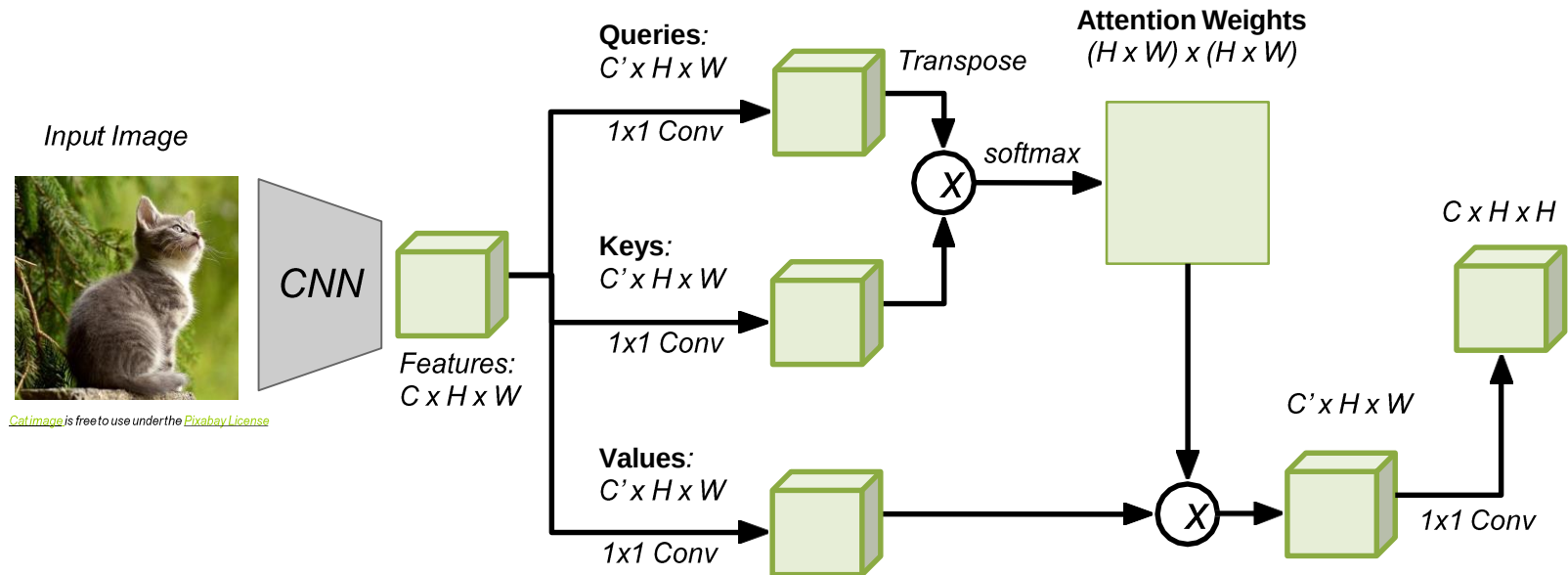
Example: CNN with Self-Attention



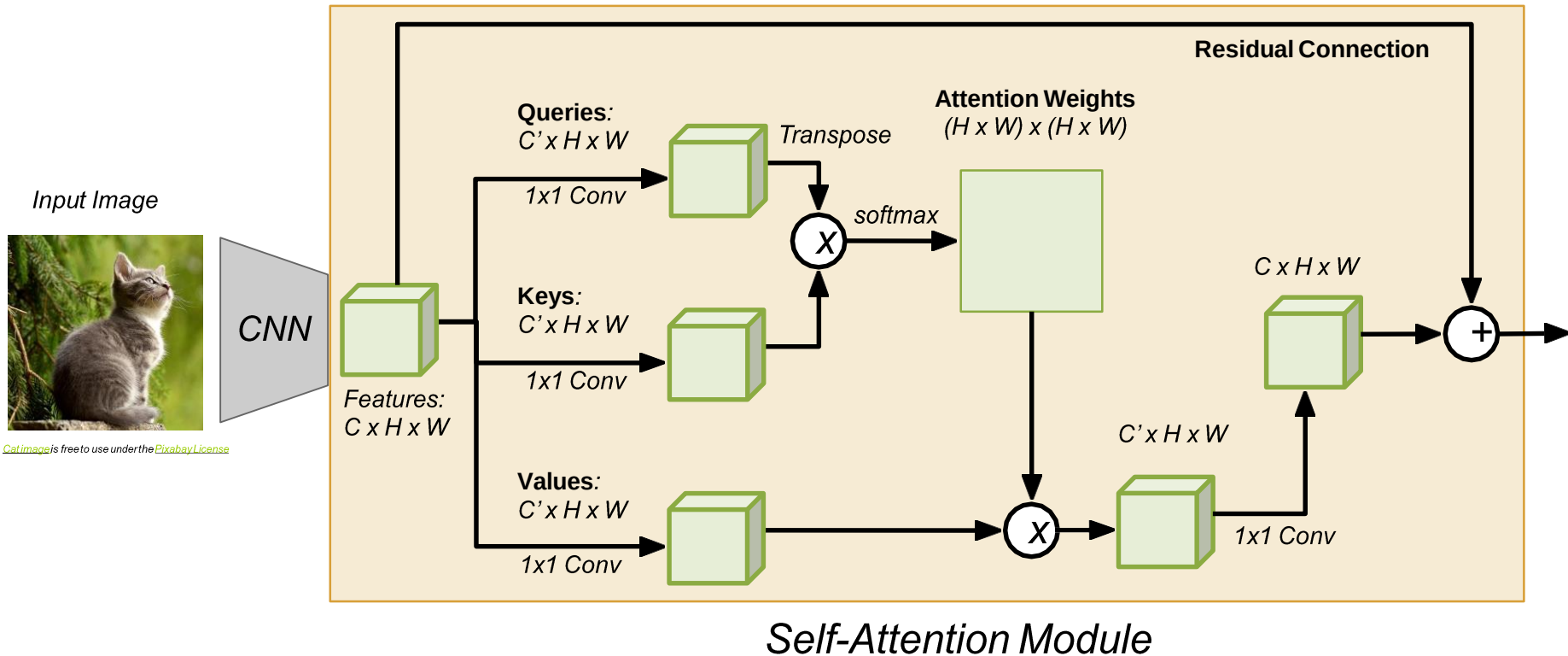
Example: CNN with Self-Attention



Example: CNN with Self-Attention



Example: CNN with Self-Attention



Comparing RNNs to Transformer

- **RNNs**
- (+) LSTMs work reasonably well for long sequences.
- (-) Expects an ordered sequences of inputs.
- (-) Sequential computation: subsequent hidden states can only be computed after the previous ones are done.
- **Transformer:**
- (+) Good at long sequences. Each attention calculation looks at all inputs.
- (+) Can operate over unordered sets or ordered sequences with positional encodings.
- (+) Parallel computation: All alignment and attention scores for all inputs can be done in parallel.
- (-) Requires a lot of memory: $N \times M$ alignment and attention scalars need to be calculated and stored for a single self-attention head. (but GPUs are getting bigger and better)



Transformer

- “ImageNet Moment for Natural Language Processing”
- **Pretraining:**
- Download a lot of text from the internet
- Train a giant Transformer model for language modeling
- **Finetuning:**
- Fine-tune the Transformer on your own NLP task

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

On the Opportunities and Risks of Foundation Models

Rishi Bommasani* Drew A. Hudson Ehsan Adeli Russ Altman Simran Arora
Sydney von Arx Michael S. Bernstein Jeannette Bohg Antoine Bosselut Emma Brunskill
Erik Brynjolfsson Shyamal Buch Dallas Card Rodrigo Castellon Niladri Chatterji
Annie Chen Kathleen Creel Jared Quincy Davis Dorottya Demszky Chris Donahue
Moussa Doumbouya Esin Durmus Stefano Ermon John Etchemendy Kawin Ethayarajh
Li Fei-Fei Chelsea Finn Trevor Gale Lauren Gillespie Karan Goel Noah Goodman
Shelby Grossman Neel Guha Tatsunori Hashimoto Peter Henderson John Hewitt
Daniel E. Ho Jenny Hong Kyle Hsu Jing Huang Thomas Icard Saahil Jain
Dan Jurafsky Pratyusha Kalluri Siddharth Karamcheti Geoff Keeling Fereshte Khani
Omar Khattab Pang Wei Koh Mark Krass Ranjay Krishna Rohith Kuditipudi
Ananya Kumar Faisal Ladhak Mina Lee Tony Lee Jure Leskovec Isabelle Levent
Xiang Lisa Li Xuechen Li Tengyu Ma Ali Malik Christopher D. Manning
Suvir Mirchandani Eric Mitchell Zanele Munyikwa Suraj Nair Avanika Narayan
Deepak Narayanan Ben Newman Allen Nie Juan Carlos Niebles Hamed Nilforoshan
Julian Nyarko Giray Ogut Laurel Orr Isabel Papadimitriou Joon Sung Park Chris Piech
Eva Portelance Christopher Potts Aditi Raghunathan Rob Reich Hongyu Ren
Frieda Rong Yusuf Roohani Camilo Ruiz Jack Ryan Christopher Ré Dorsa Sadigh
Shiori Sagawa Keshav Santhanam Andy Shih Krishnan Srinivasan Alex Tamkin
Rohan Taori Armin W. Thomas Florian Tramèr Rose E. Wang William Wang Bohan Wu
Jiajun Wu Yuhuai Wu Sang Michael Xie Michihiro Yasunaga Jiaxuan You Matei Zaharia
Michael Zhang Tianyi Zhang Xikun Zhang Yuhui Zhang Lucia Zheng Kaitlyn Zhou
Percy Liang*¹

Center for Research on Foundation Models (CRFM)
Stanford Institute for Human-Centered Artificial Intelligence (HAI)
Stanford University

Image Captioning using Transformers

- **Input:** Image I
- **Output:** Sequence $y = y_1, y_2, \dots, y_T$

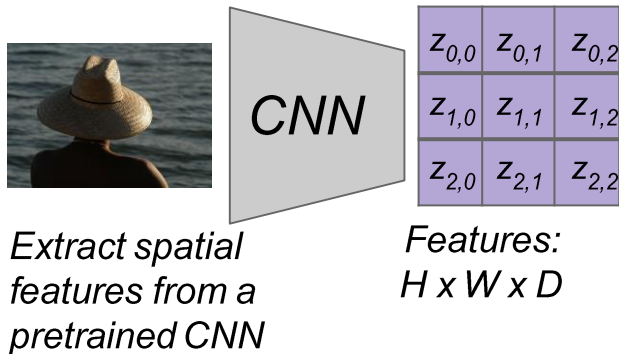


Image Captioning using Transformers

- **Input:** Image I
- **Output:** Sequence $y = y_1, y_2, \dots, y_T$

Encoder: $c = T_w(z)$
where z is spatial CNN features
 $T_w(\cdot)$ is the transformer encoder

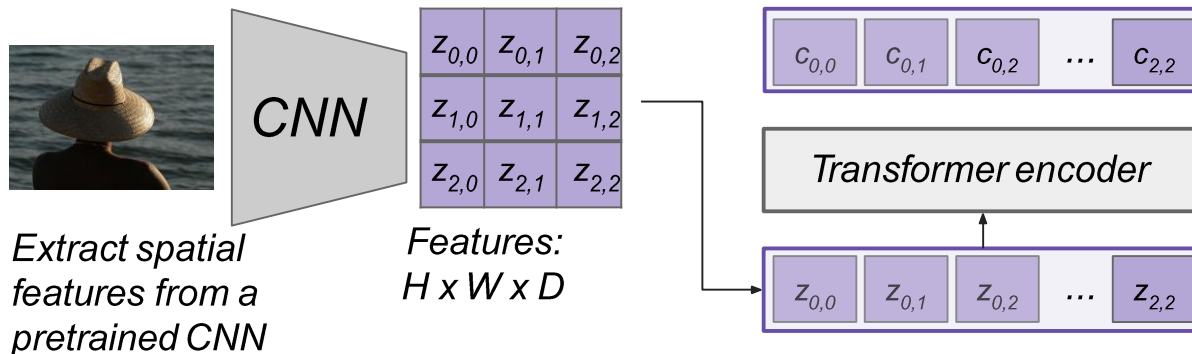
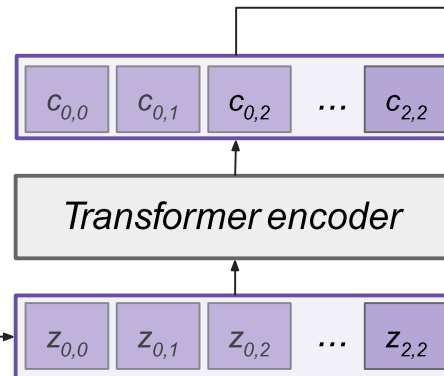
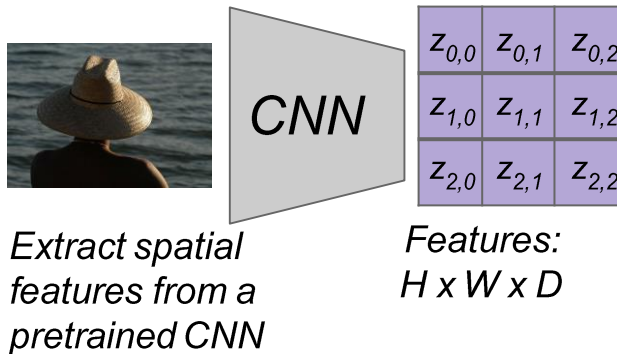


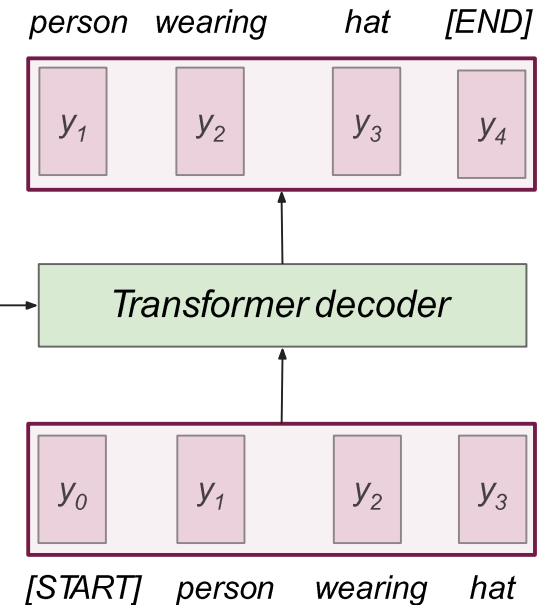
Image Captioning using Transformers

- **Input:** Image I
- **Output:** Sequence $y = y_1, y_2, \dots, y_T$

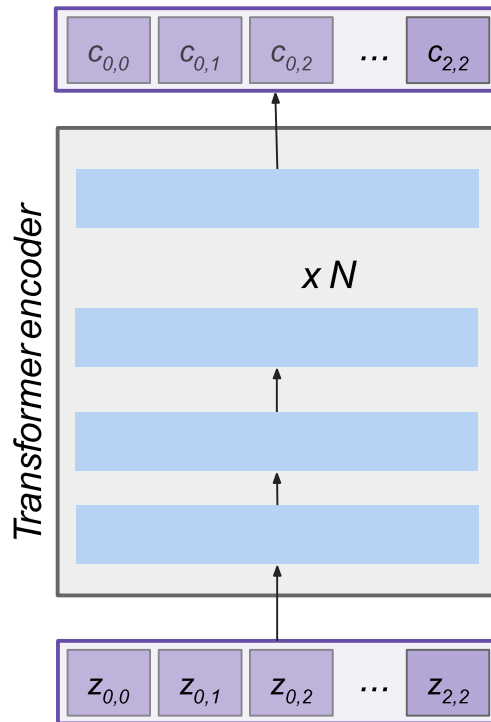
Encoder: $\mathbf{c} = T_w(\mathbf{z})$
 where $\mathbf{z}$ is spatial CNN features
 $T_w(\cdot)$ is the transformer encoder



Decoder: $y_t = T_D(\mathbf{y}_{0:t-1}, \mathbf{c})$
 where $T_D(\cdot)$ is the transformer decoder



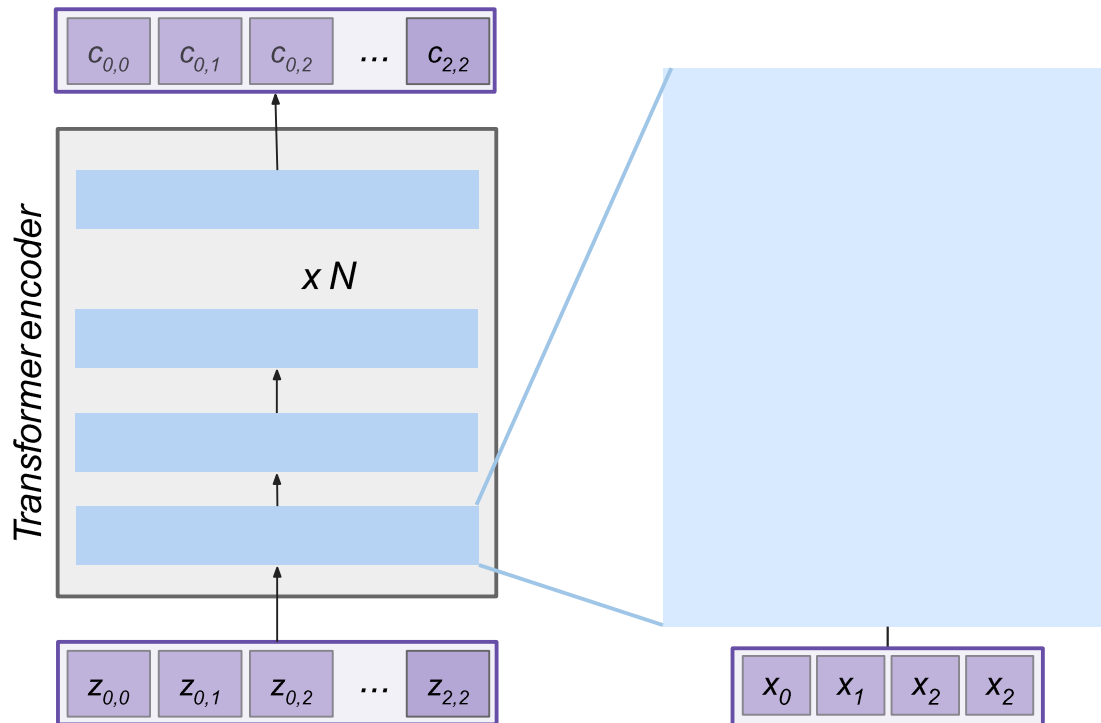
The Transformer encoder block



Made up of N encoder blocks.

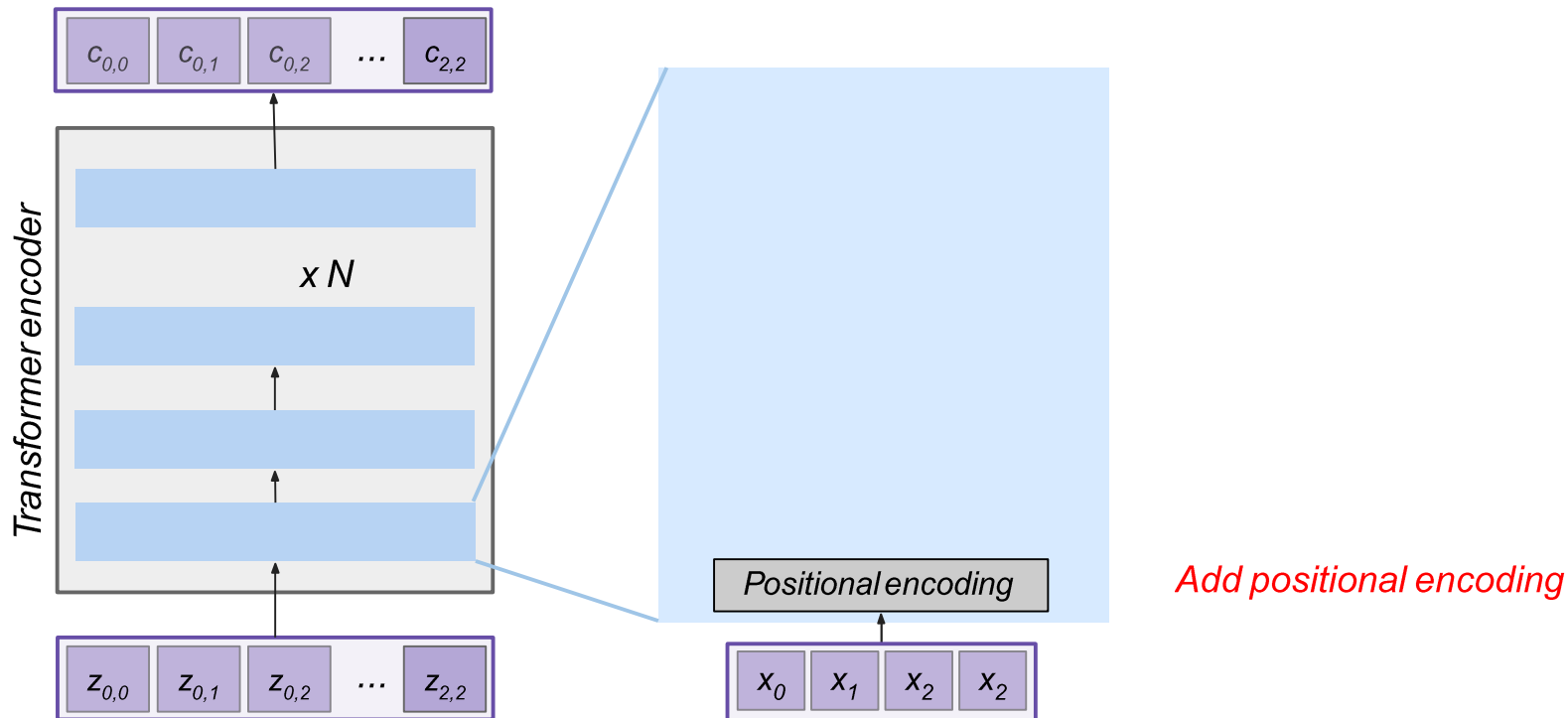
In vaswani et al. $N = 6$, $D_q = 512$

The Transformer encoder block

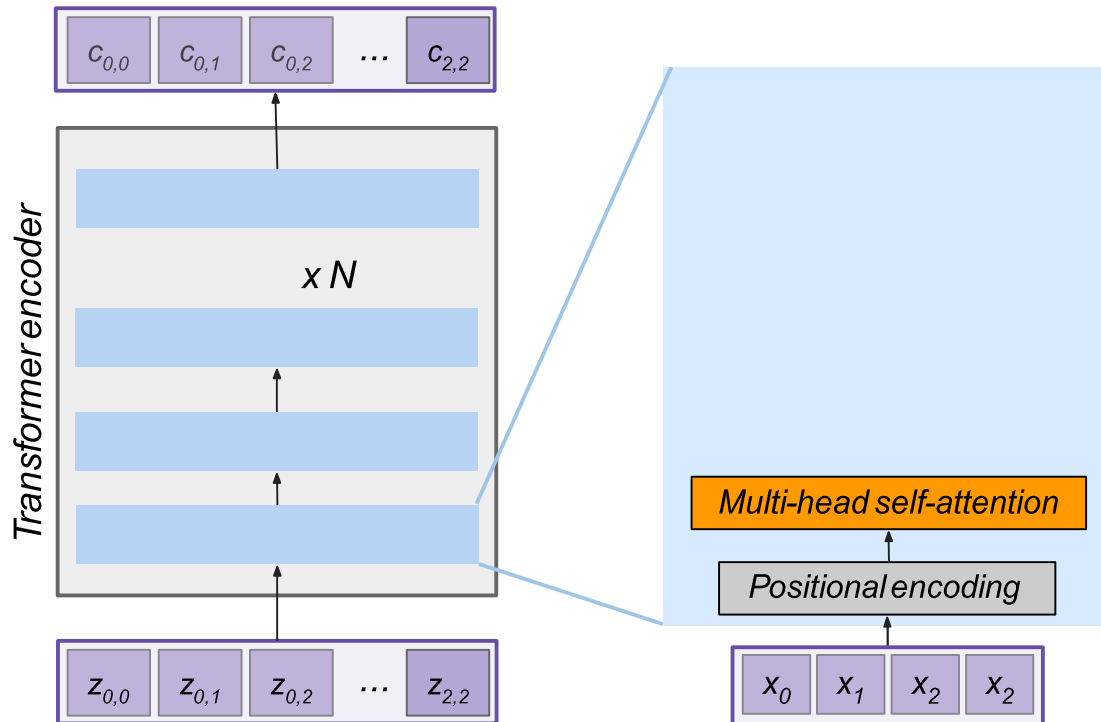


Let's dive into one encoder block

The Transformer encoder block



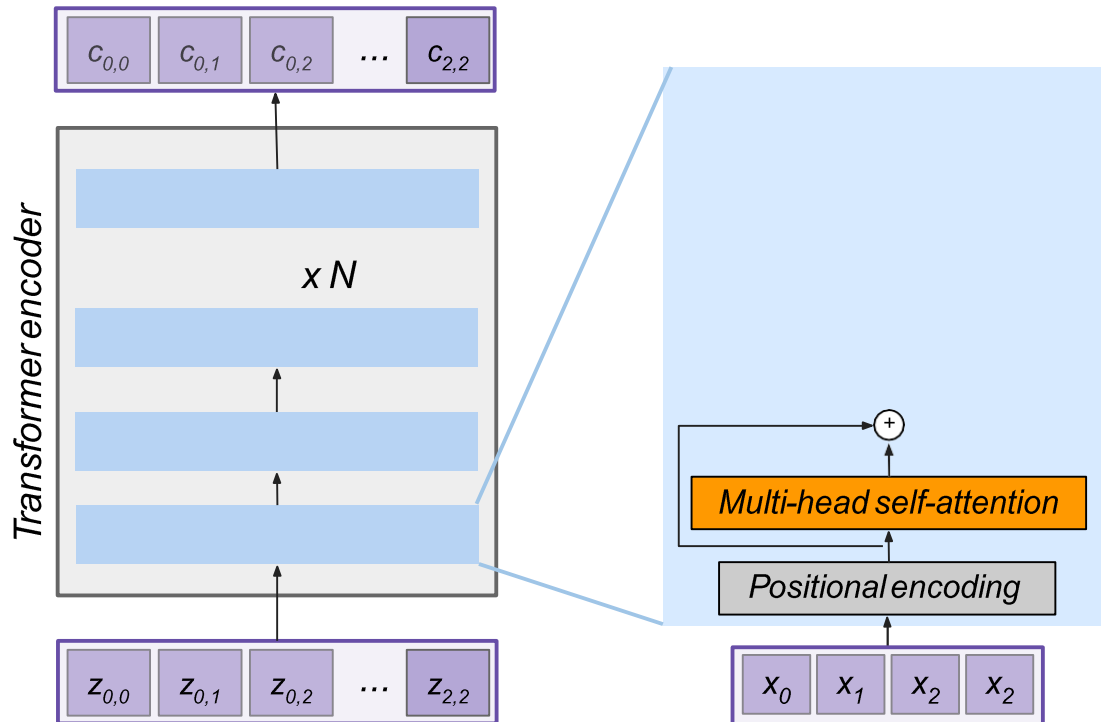
The Transformer encoder block



Attention attends over all the vectors

Add positional encoding

The Transformer encoder block

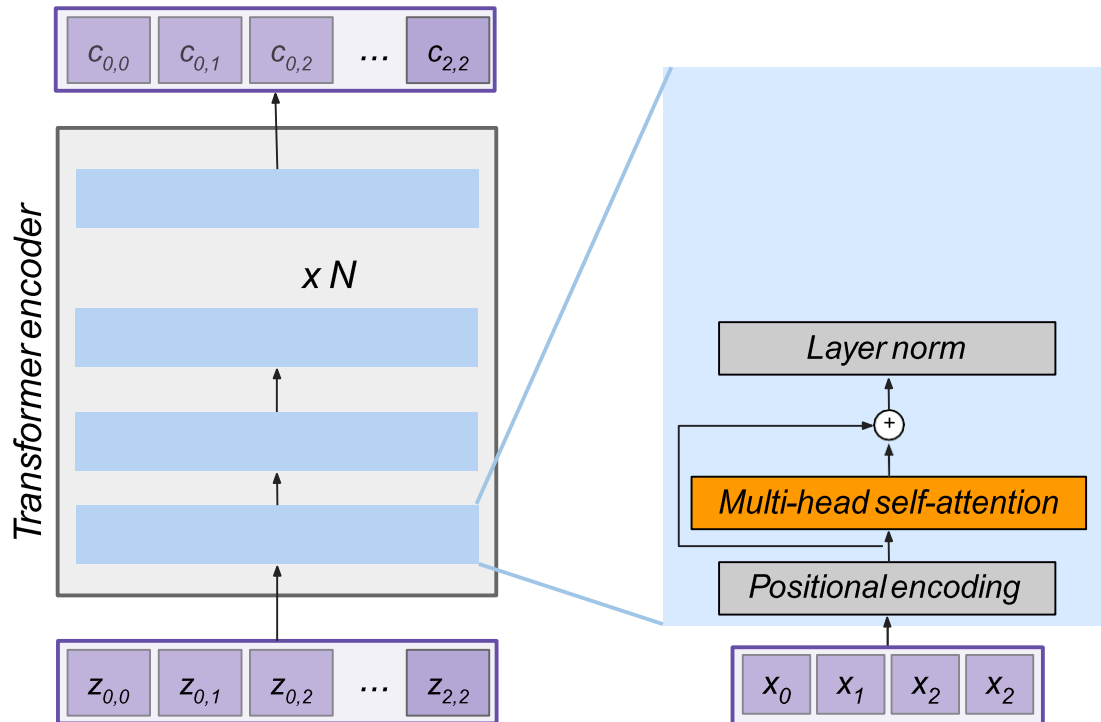


Residual connection

Attention attends over all the vectors

Add positional encoding

The Transformer encoder block



Batch Normalization

batch

1	3	6
2	2	2
0	1	5
4	6	1
5	2	3
1	0	1

Same for all training examples

mean	std
3	3
2	0
3	3
4	3
3	2
1	1

Layer Normalization

batch

1	3	6
2	2	2
0	1	5
4	6	1
5	2	3
1	0	1

mean

2	3	3
2	2	2

std

2	2	2
---	---	---

Same for all feature dimensions

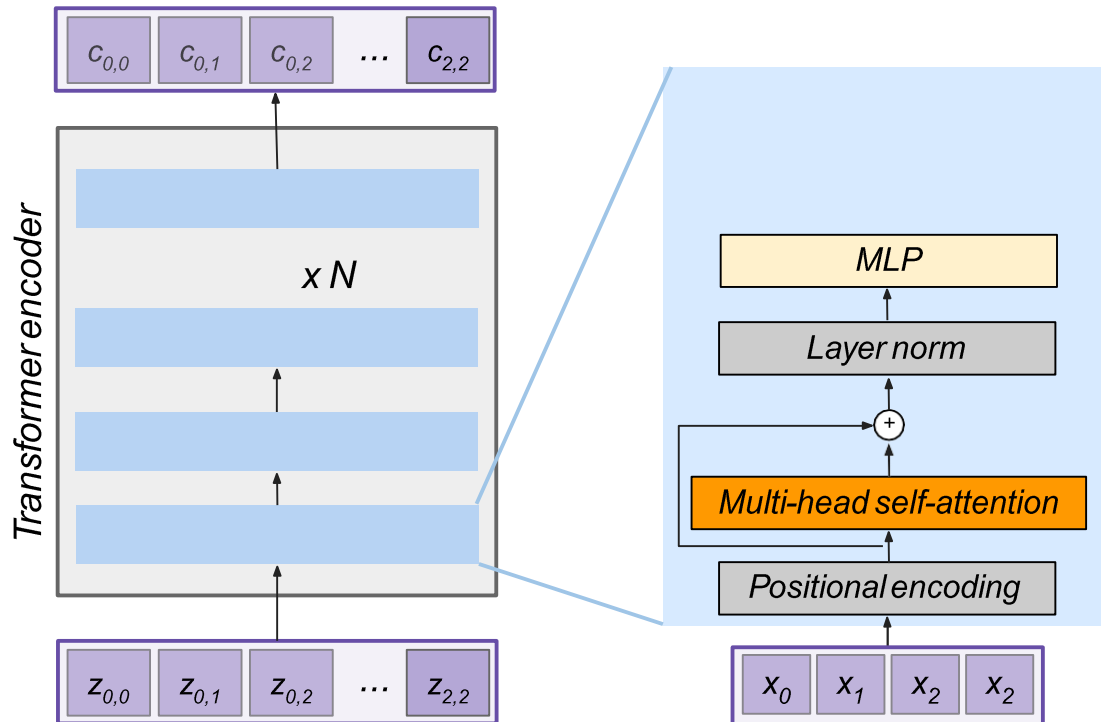
LayerNorm over each vector individually

Residual connection

Attention attends over all the vectors

Add positional encoding

The Transformer encoder block



MLP over each vector individually

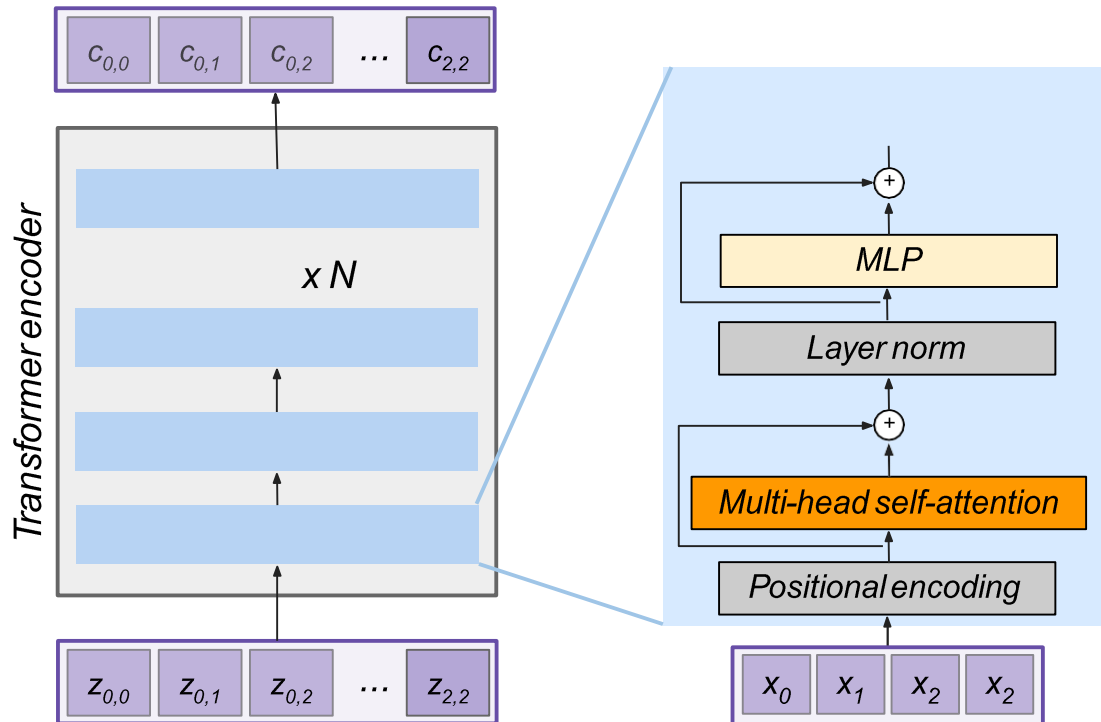
LayerNorm over each vector individually

Residual connection

Attention attends over all the vectors

Add positional encoding

The Transformer encoder block



Residual connection

MLP over each vector individually

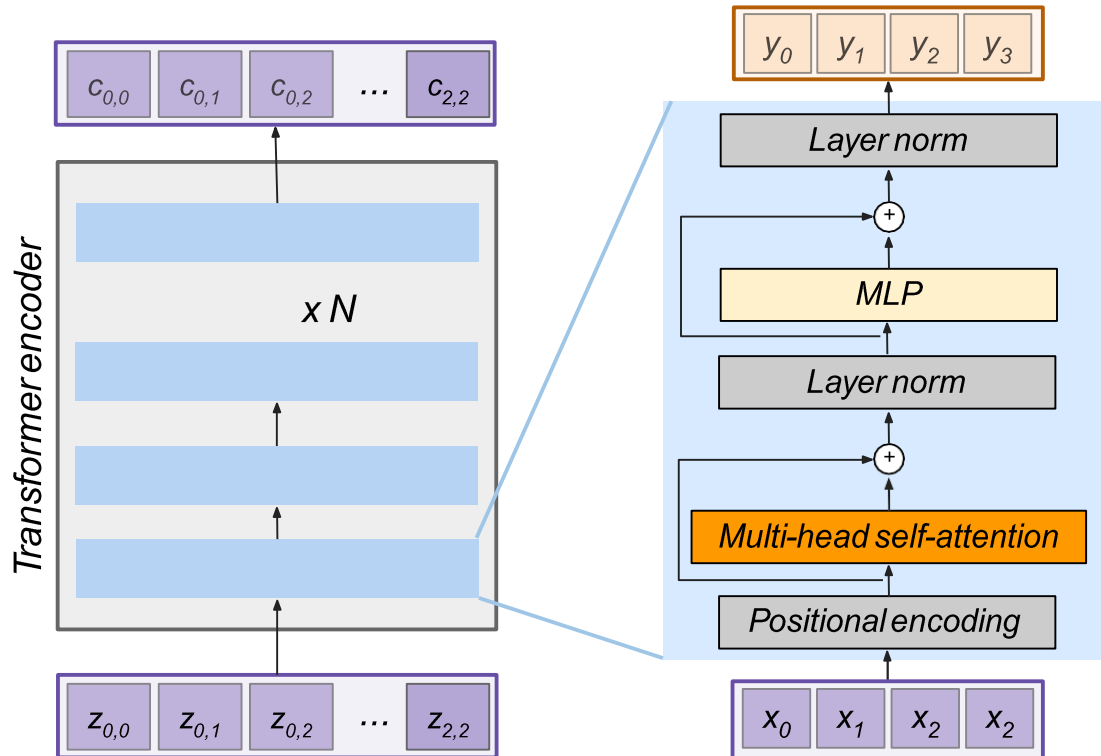
LayerNorm over each vector individually

Residual connection

Attention attends over all the vectors

Add positional encoding

The Transformer encoder block



Transformer Encoder Block:

Inputs: Set of vectors x

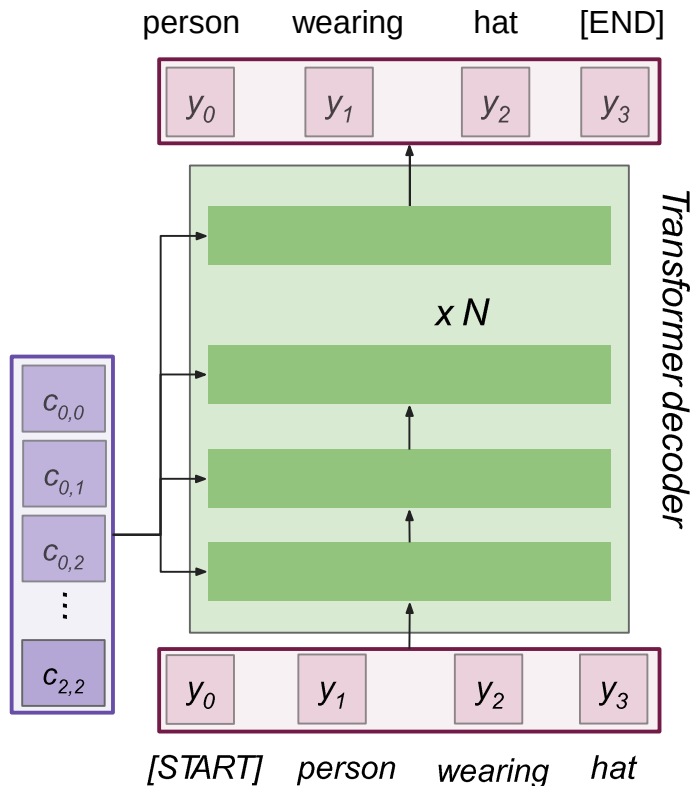
Outputs: Set of vectors y

Self-attention is the only interaction between vectors.

Layer norm and MLP operate independently per vector.

Highly scalable, highly parallelizable, but high memory usage.

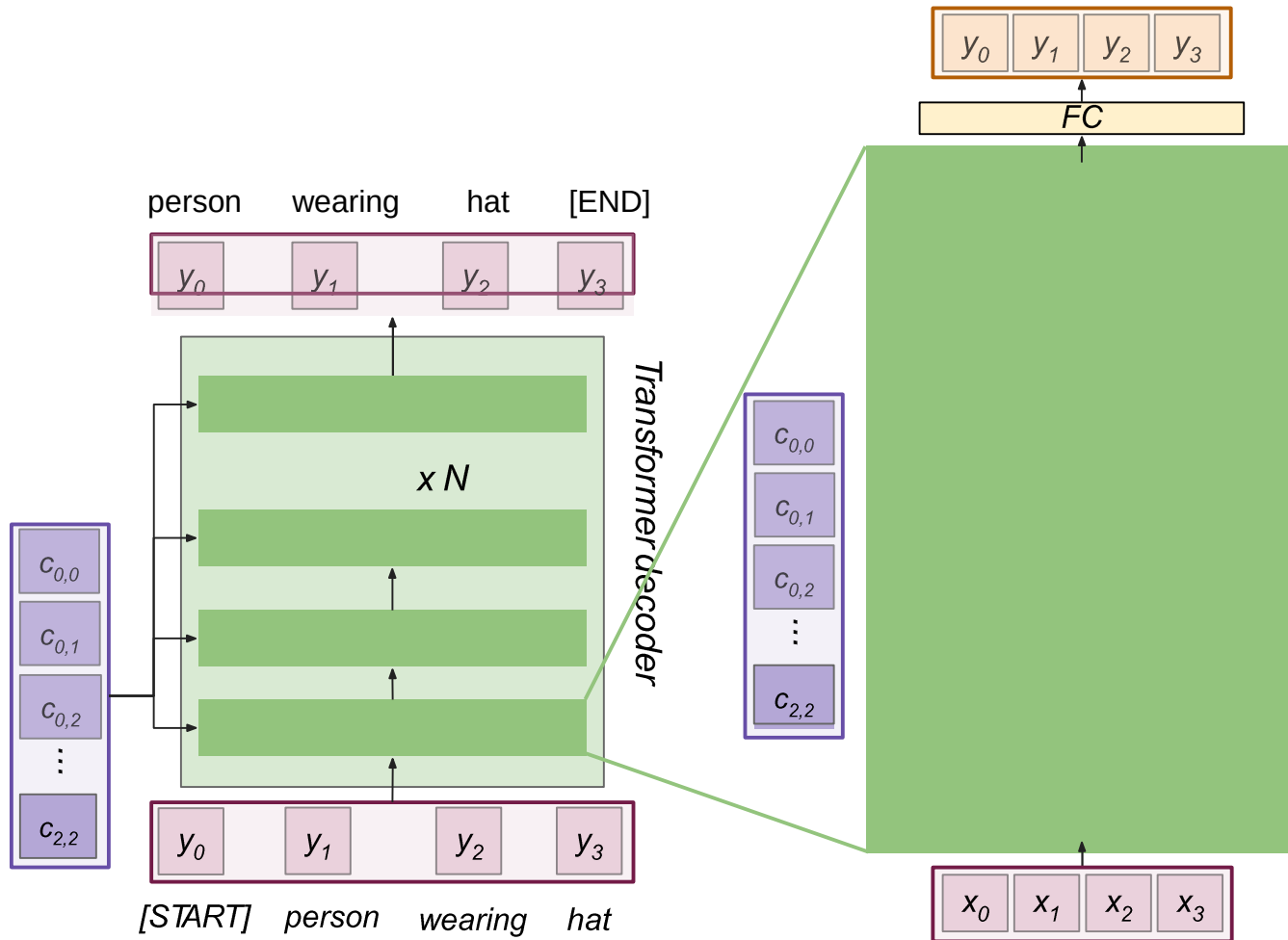
The Transformer Decoder block



Made up of N decoder blocks.

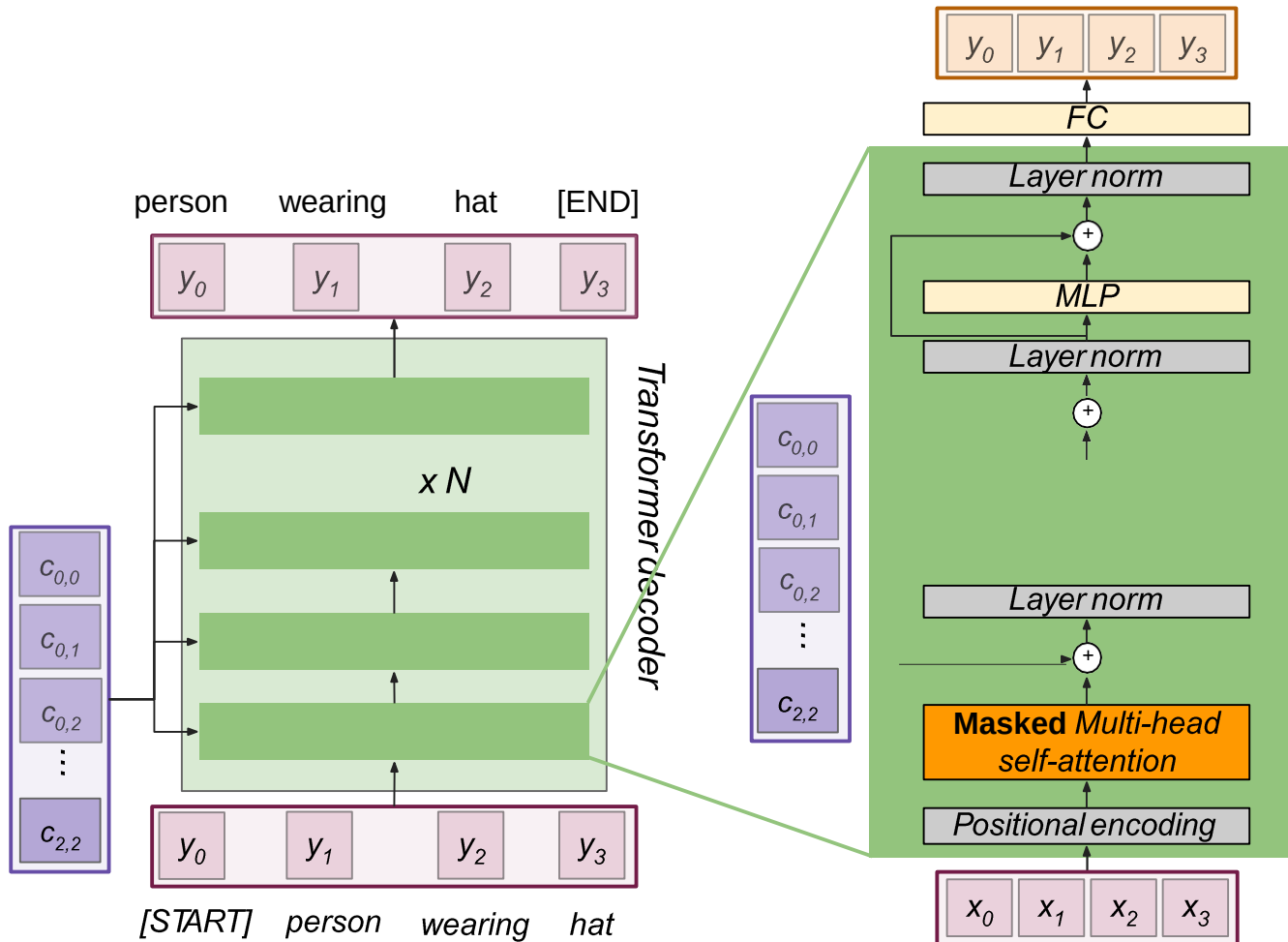
In vaswani et al. $N = 6, D_q = 512$

The Transformer Decoder block



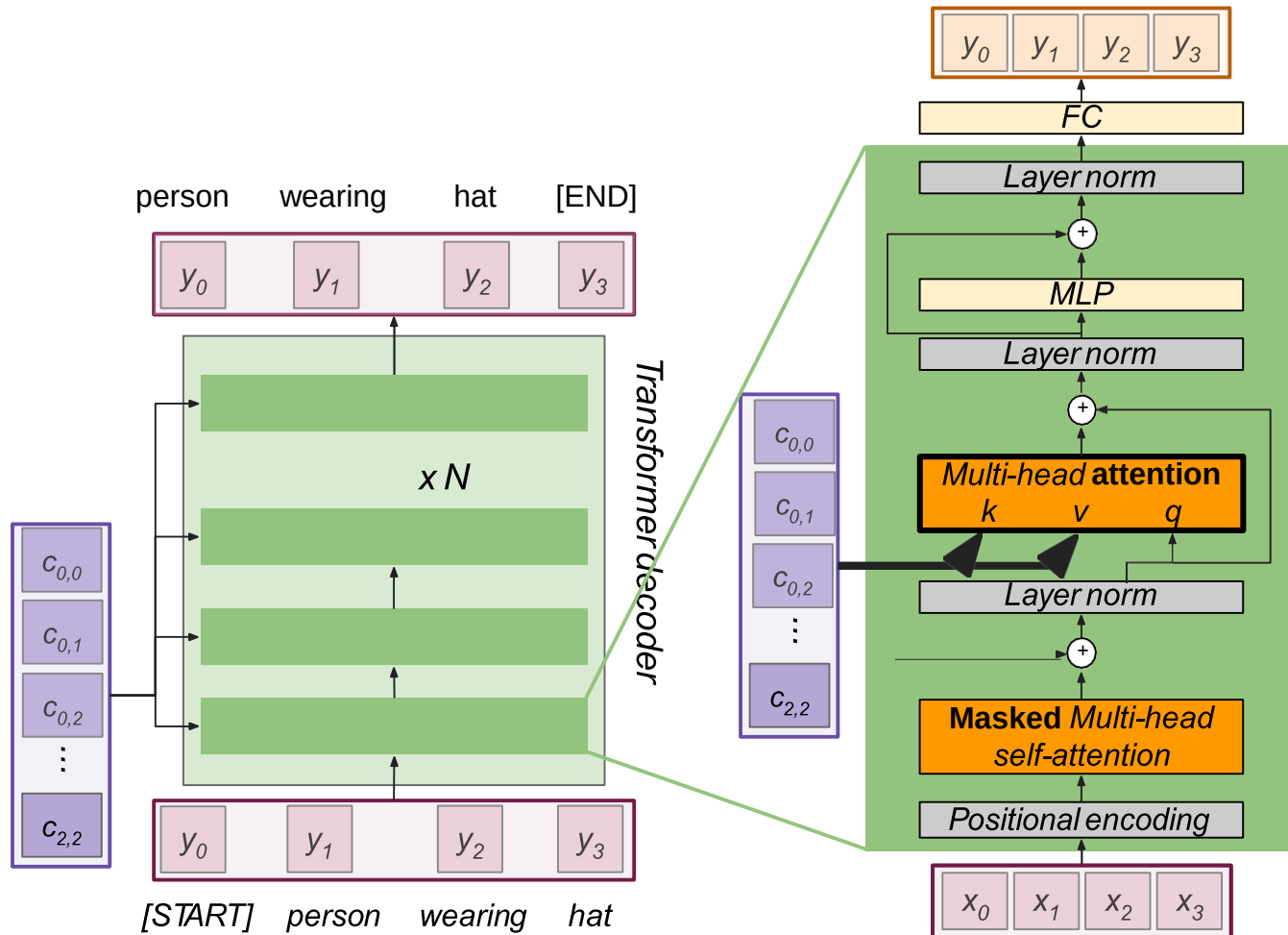
Let's dive into the transformer decoder block

The Transformer Decoder block



Most of the network is the same the transformer encoder.

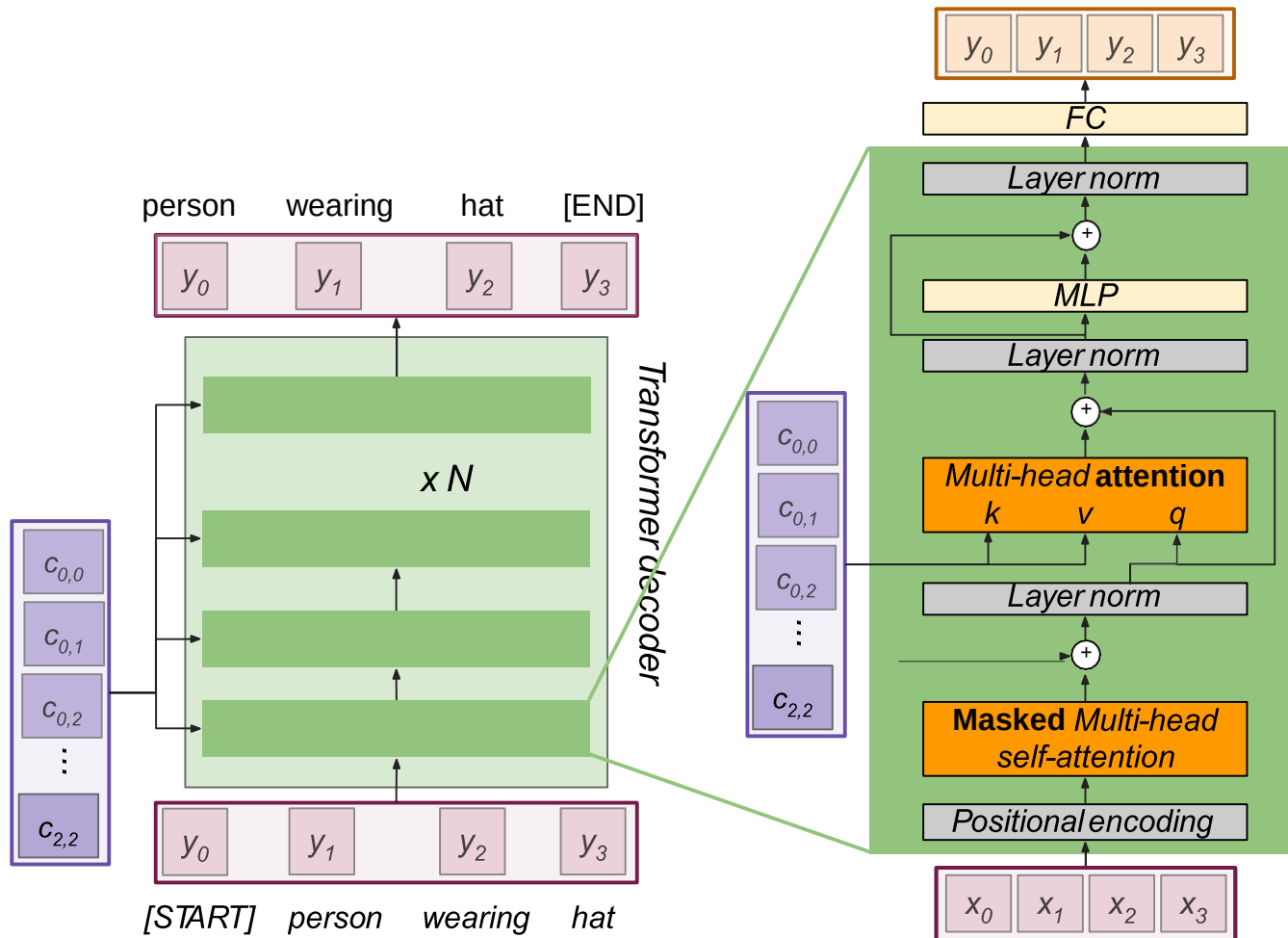
The Transformer Decoder block



Multi-head attention block attends over the transformer encoder outputs.

For image captions, this is how we inject image features into the decoder.

The Transformer Decoder block



Transformer Decoder Block:

Inputs: Set of vectors x and Set of context vectors c .

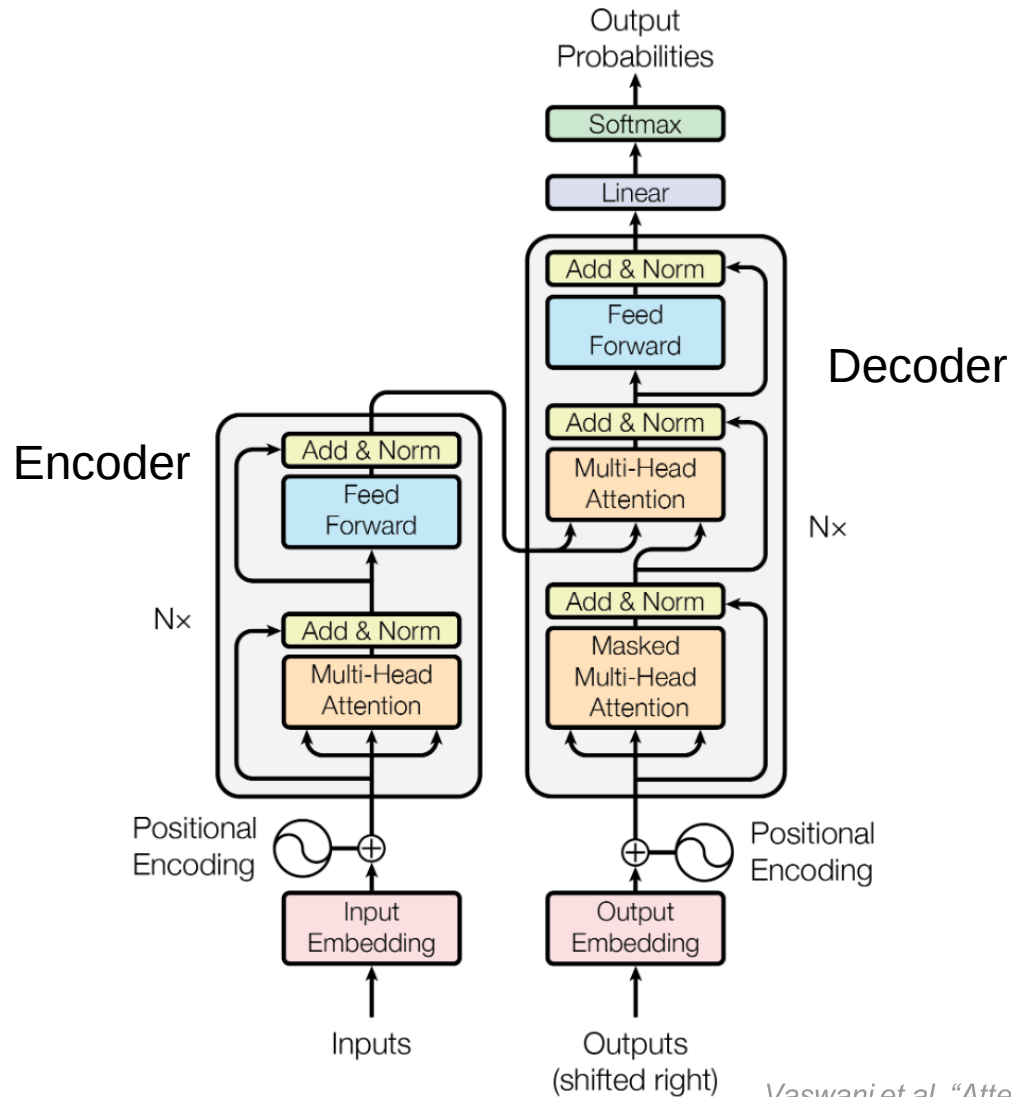
Outputs: Set of vectors y .

Masked Self-attention only interacts with past inputs.

Multi-head attention block is NOT self-attention. It attends over encoder outputs.

Highly scalable, highly parallelizable, but high memory usage.

The Transformer Architecture



Vaswani et al, "Attention is all you need", NeurIPS 2017

Image Captioning using Transformers

- No recurrence at all

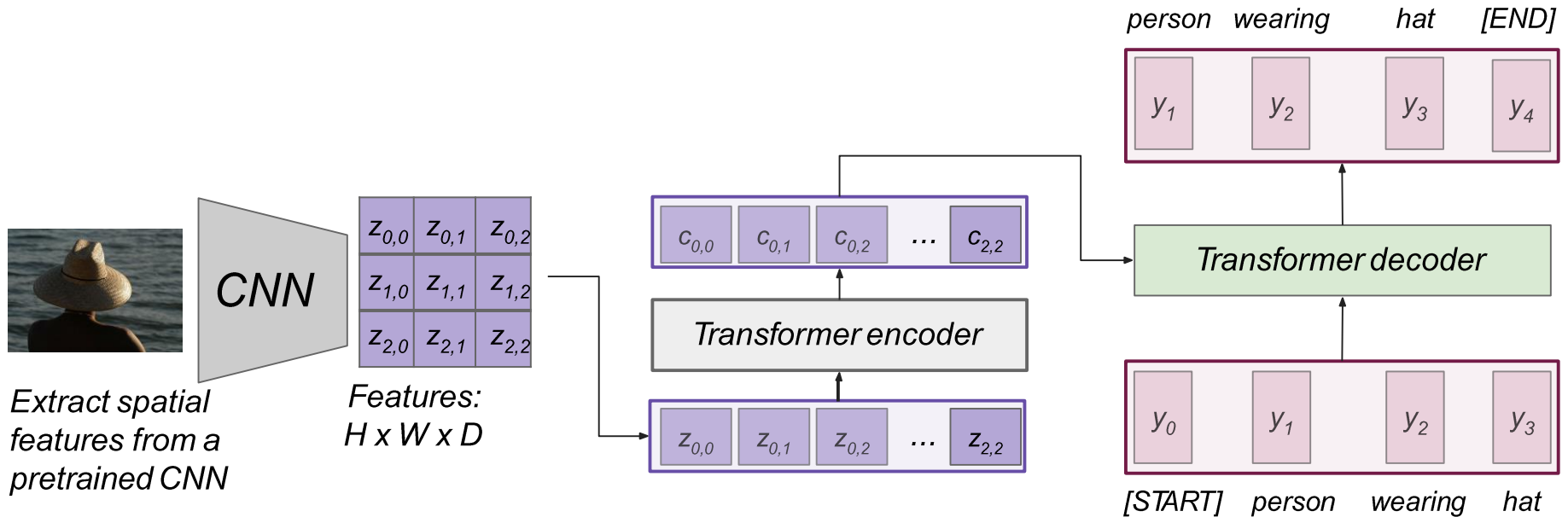


Image Captioning using Transformers

- Perhaps we don't need convolutions at all?

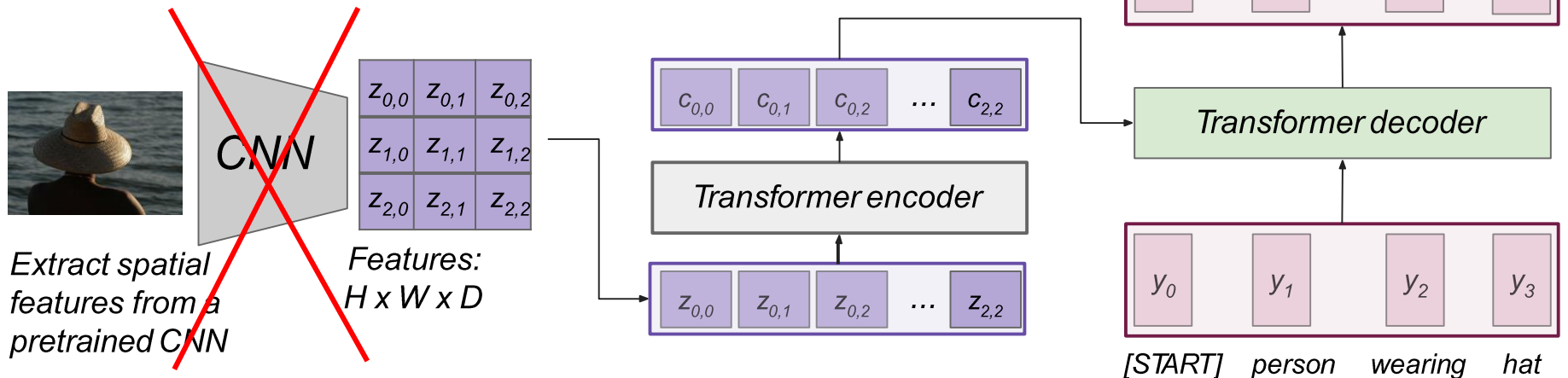
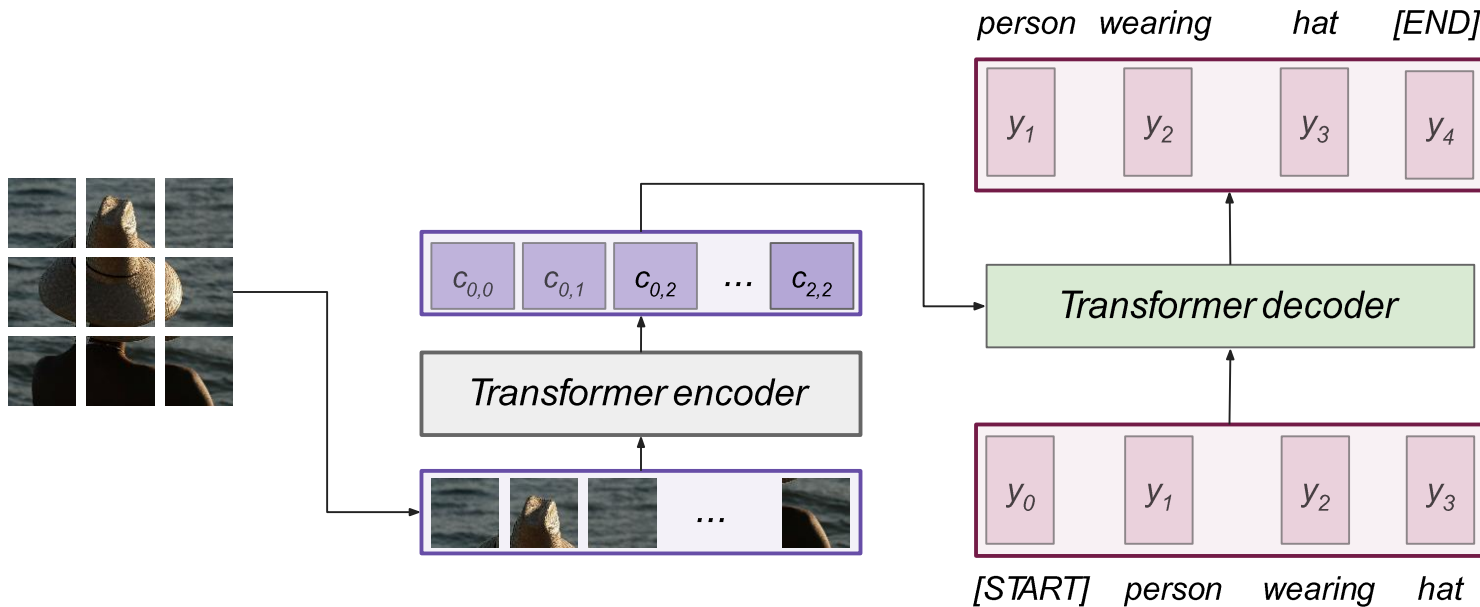


Image Captioning using **ONLY** transformers

- Transformers from pixels to language



Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ArXiv 2020 [Colab link](#) to an implementation of vision transformers

Vision Transformers vs. ResNets

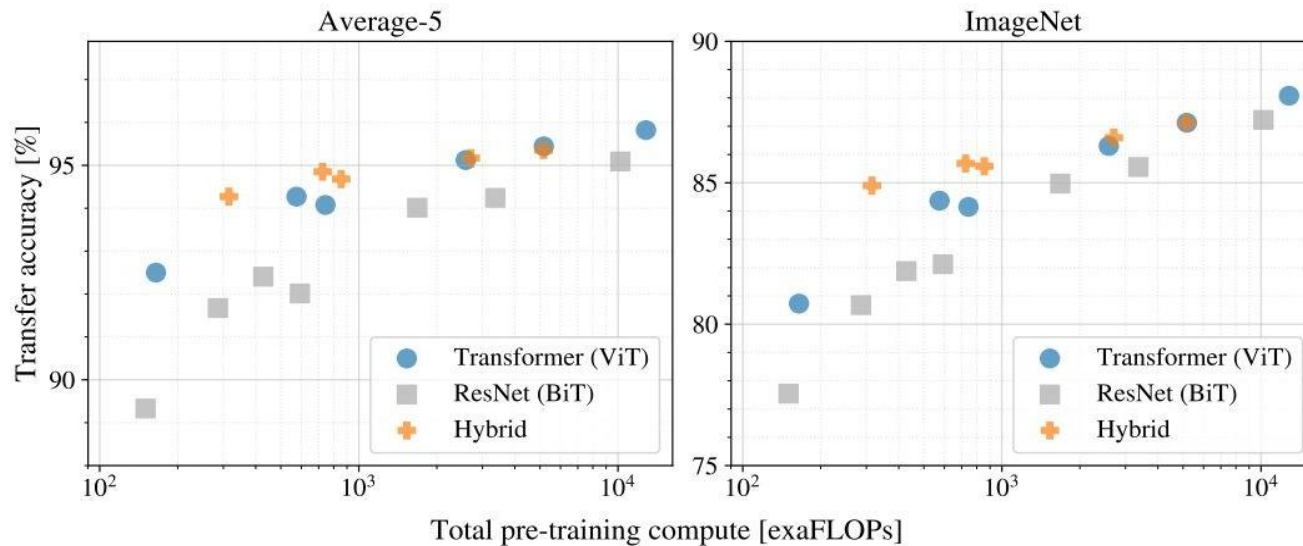
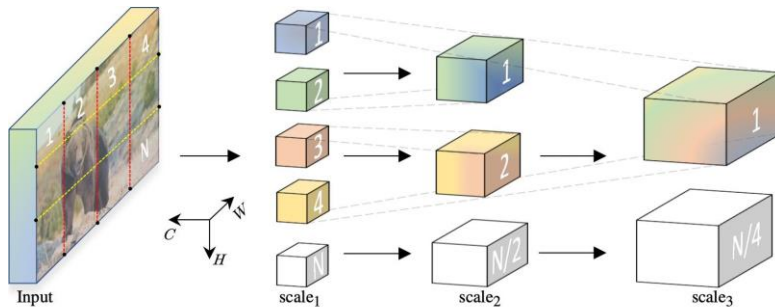


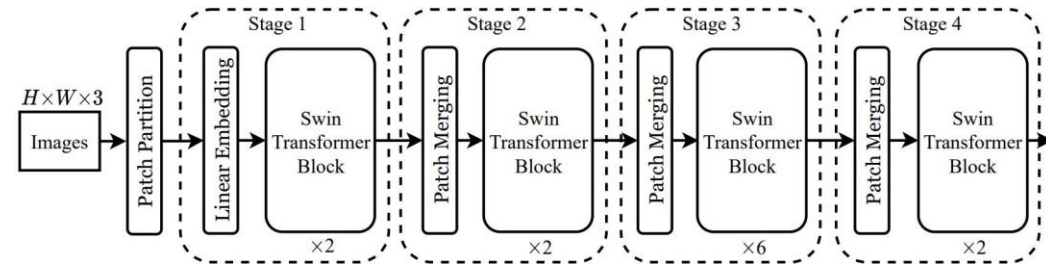
Figure 5: Performance versus cost for different architectures: Vision Transformers, ResNets, and hybrids. Vision Transformers generally outperform ResNets with the same computational budget. Hybrids improve upon pure Transformers for smaller model sizes, but the gap vanishes for larger models.

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ArXiv 2020 [Colab link](#) to an implementation of vision transformers

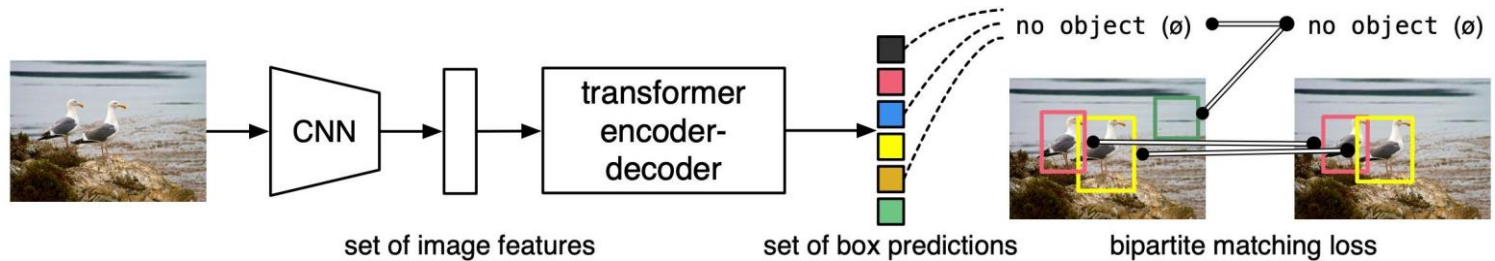
Vision Transformers



Fan et al, "Multiscale Vision Transformers", ICCV 2021

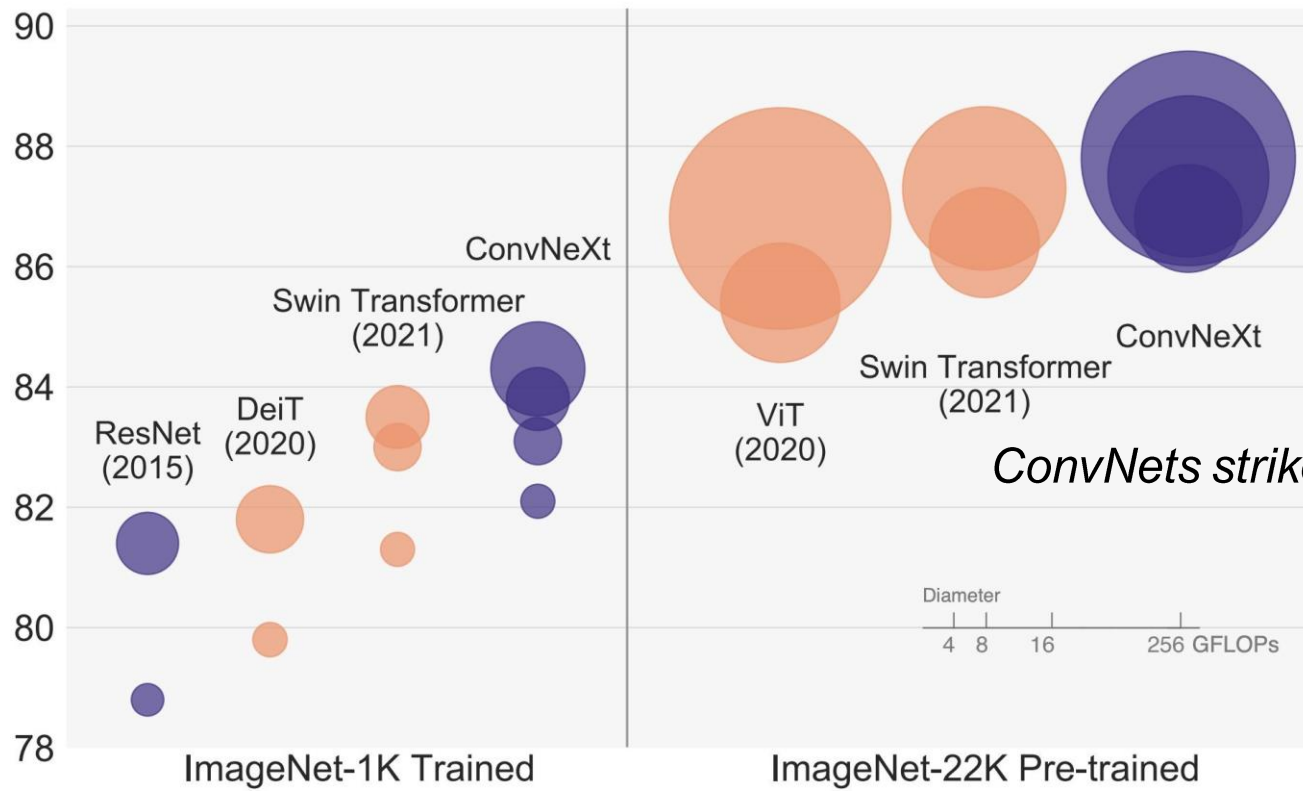


Liu et al, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows", ICCV 2021



Carion et al, "End-to-End Object Detection with Transformers", ECCV 2020

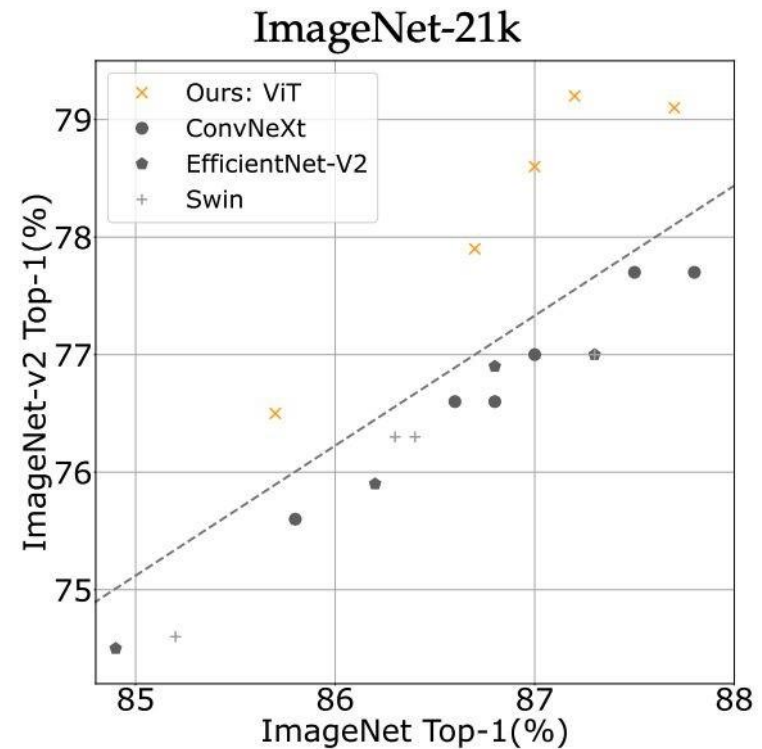
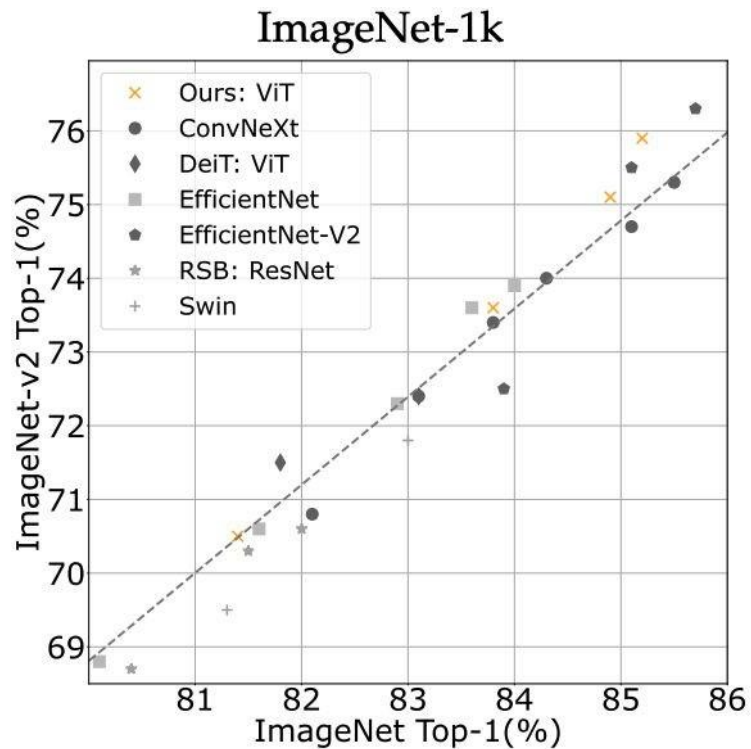
ImageNet-1K Acc.



ConvNets strike back!

DeiT III: Revenge of the ViT

Hugo Touvron^{*,†} Matthieu Cord[†] Hervé Jégou^{*}



Summary

- 📖 Adding **attention** to RNNs allows them to "attend" to different parts of the input at every time step
- 📖 The **general attention layer** is a new type of layer that can be used to design new neural network architectures
- 📖 **Transformers** are a type of layer that uses **self-attention** and layer norm.
 - *It is highly **scalable** and highly **parallelizable***
 - ✦ **Faster** training, **larger** models, **better** performance across vision and language tasks
 - ✦ They are quickly replacing RNNs, LSTMs, and may(?) even replace convolutions.